



第八章

AT89系列单片机的接口扩展技术

刘剑锋

ljf@csu.edu.cn





主要内容

8.1 I/O接口的扩展技术

8.2 键盘及其与单片机的接口技术

8.3 LED显示器及其与单片机的接口技术

8.4 LCD显示器及其接口技术

8.5 A/D、D/A转换器及其与单片机的接口技术

8.1 I/O接口的扩展技术

◇ I/O接口的功能

◆ 实现和不同外设的速度匹配

多种多样外设的工作速度差别很大，但大多数外设的速度很慢，无法和 μs 量级的单片机速度相比。单片机和外设之间的数据传送方式有同步、异步、中断三种。需要I/O接口电路与外设之间传送状态信息，以实现单片机与外设之间的速度匹配。

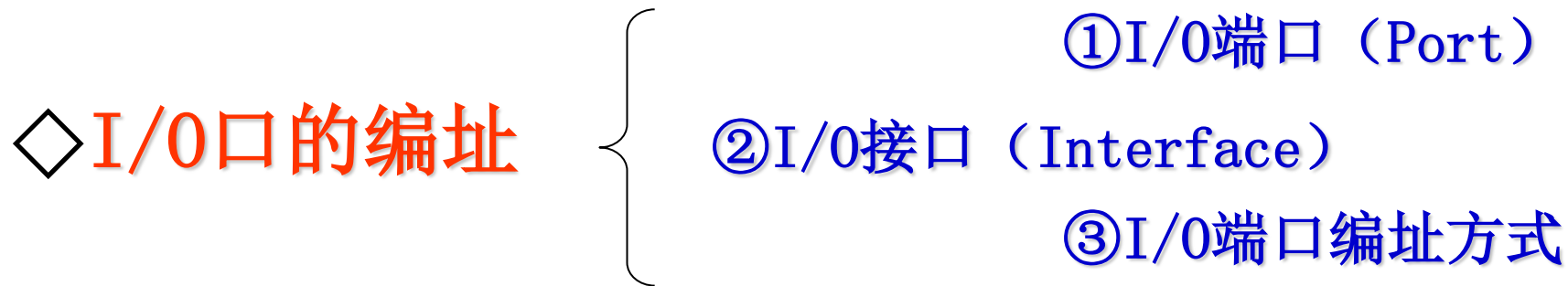
◆ 输出数据锁存

由于单片机工作速度快，数据在数据总线上保留的时间十分短暂，无法满足慢速外设的数据接收。I/O电路应具有数据锁存器，以保证输出数据能被接收设备所接收。

◆ 输入数据三态缓冲

输入设备向单片机输入数据时，要经过数据总线，但数据总线上面可能“挂”有多个数据源，为了传送数据时不发生冲突，只允许当前时刻正在进行数据传送的数据源使用数据总线，其余的数据应处于隔离状态，为此要求接口电路能为数据输入提供三态缓冲功能。

8.1 I/O接口的扩展技术



◆ I/O端口 (Port)

I/O端口简称I/O口，常指I/O接口电路中具有端口地址的寄存器或缓冲器。

◆ I/O接口 (Interface)

I/O接口是指单片机与外设的I/O接口芯片。一个I/O接口芯片可以有多个I/O端口，传送数据的称为数据口，传送命令的称为命令口，传送状态的称为状态口。

◆ I/O端口编址方式

常用的I/O端口编址有两种方式，一种是统一编址方式（或称为存储器映像编址），另一种是独立编址方式。

▲ 统一编址方式

统一编址就是I/O端口的寄存器与存储器单元同等对待，统一进行编址，把存储器的一部分地址空间分给端口，把每一个端口作为一个存储单元。

统一编址的优点是对端口信息的处理就像对主存储器单元一样，不必专门设置专门的输入 / 输出指令来访问端口，直接使用访问数据存储器的指令进行I/O操作，简单、方便且功能强。但是，统一编址会减少存储器容量。

▲ 独立编址方式

独立编址就是I/O地址空间和存储器地址空间分开编址，端口不占存储器地址空间。

独立编址的优点是I/O地址空间和存储器地址空间相互独立，界限分明。但是，必须设置专门的输入 / 输出指令访问端口。访问存储器与访问端口采用不同的指令，译码后，产生的控制信息不同，其地址虽有重叠，但不会发生冲突。



8.1 I/O接口的扩展技术

◇ I/O接口数据的传送方式

- ①无条件传送方式;
- ②查询传送方式;
- ③中断传送方式.

◆无条件传送方式

无条件传送又称为同步传送。当外设时刻都处于“准备好”状态，外设的速度可与单片机速度相比拟时，常采用同步传送方式，这种方式不需要交换状态信息。例如，将数据输出给LED数码管，一般采用这种传送方式。

8.1 I/O接口的扩展技术

◆ 查询传送方式

查询传送又称为有条件传送，也称为异步传送。在查询传送方式中，单片机首先要查询外设是否准备好，只有当外设准备好后，再进行数据传送。

查询方式的过程为：查询→等待→数据传送。

查询传送的优点是通用性好，可用于各种速度的外设和单片机之间的数据传送，硬件连线和查询程序十分简单。

其缺点是效率不高，在连续传送数据时，每传送一个数据，都有一个等待过程，等待期间CPU不能进行其他操作，CPU利用率低。

为了提高单片机的工作效率，通常采用中断传送方式。

◆ 中断传送方式

中断传送方式是利用AT89系列单片机本身的中断功能和I/O接口的中断功能来实现I/O数据的传送。

在这种方式中，CPU不再进行查询，只有在外设准备好后，发出数据传送请求，才中断主程序，而进入与外设进行数据传送的中断服务程序，进行数据的传送。因此，采用中断传送方式可以大大提高单片机的工作效率。



◇简单I/O接口的扩展

利用74LS系列TTL电路或CMOS电路锁存器、三态门电路作为I / O端口扩展芯片。

这种I/O端口一般都是通过P0口扩展，它具有电路简单、成本低、配置灵活的优点。

◆一种扩展简单I / O端口的实例



8.1 I/O接口的扩展技术

◆一种扩展简单I / O端口的实例

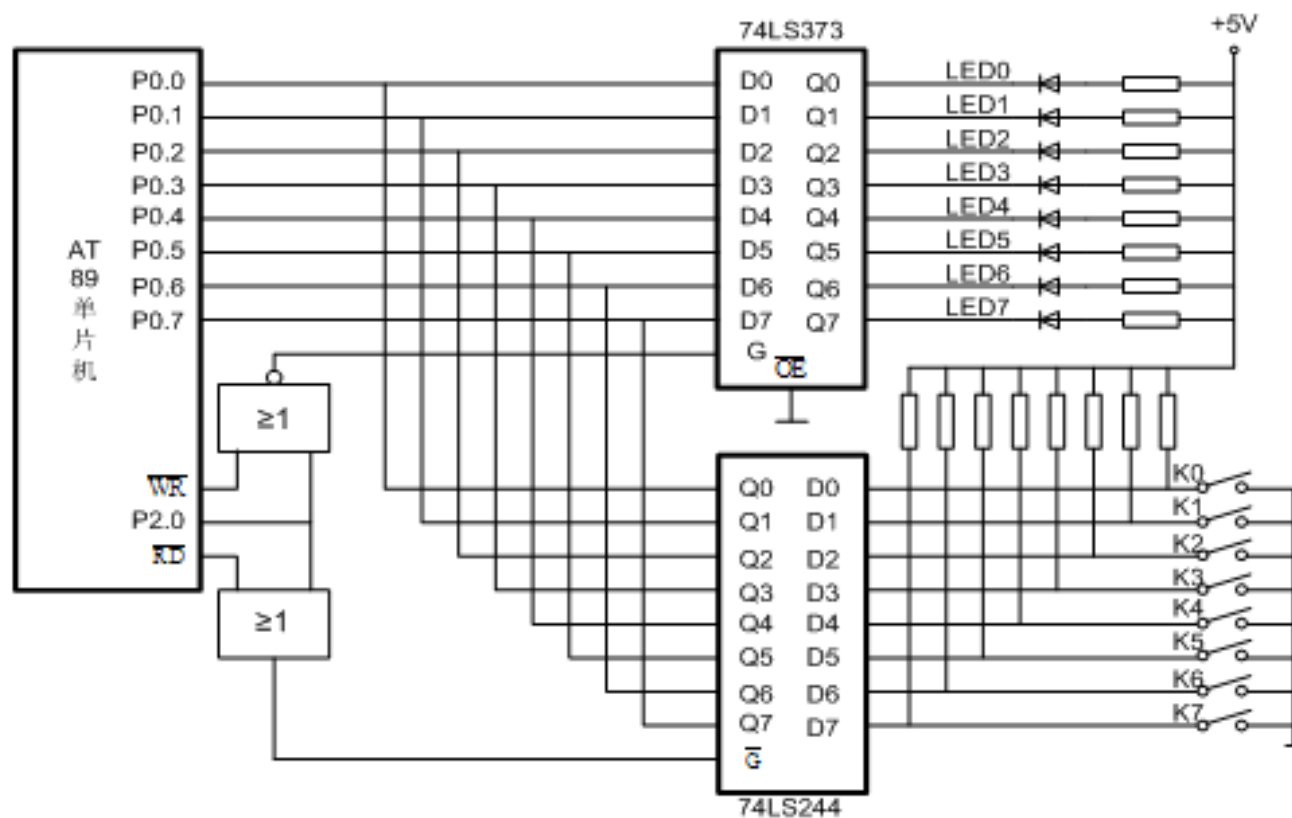


图 8-1 简单 I/O 接口扩展电路

电路的功能是按下任意键，对应的LED发光。

汇编语言程序:

LOOP: MOV DPTR, #0FEFFH	; 扩展I/O口地址送DPTR
MOVX A, @DPTR	; 通过74LS244读入数据, 检测键的状态
MOVX @DPTR, A	; 向74LS373输出数据, 驱动LED
SJMP LOOP	; 循环

C语言程序:

```
#include<reg51.h>
#include<absacc.h>
unsigned char i;
main()
{
while(1)                                // 循环
{
    i=XBYTE[0xfeff];    //通过74LS244读入数据, 检测键的状态
    XBYTE[0xfeff]=i;    // 向74LS373输出数据, 驱动LED
}
}
```


8.1 I/O接口的扩展技术

◇ 可编程8255A的并行I/O扩展

- ◆ 8255A芯片介绍
- ◆ 8255A控制字
- ◆ 8255A的3种工作方式
- ◆ 8255A和AT89系列单片机的接口

◆ 8255A芯片介绍

8255A是Intel公司生产的可编程并行I/O接口芯片，具有3个8位的并行I/O口，3种工作方式，8255A可编程并行I/O接口芯片与一般接口芯片（如前面介绍的74LS244/74LS373）一样可对单片机的I/O口进行扩展，**但8255A可通过编程改变其功能**，因而使用灵活方便，通用性强。

(一) 8255A内部结构

8255A内部由4部分组成：

- 1) 并行I/O端口A、B、C
- 2) A组和B组控制器
- 3) 数据总线缓冲器
- 4) 读/写控制逻辑电路线

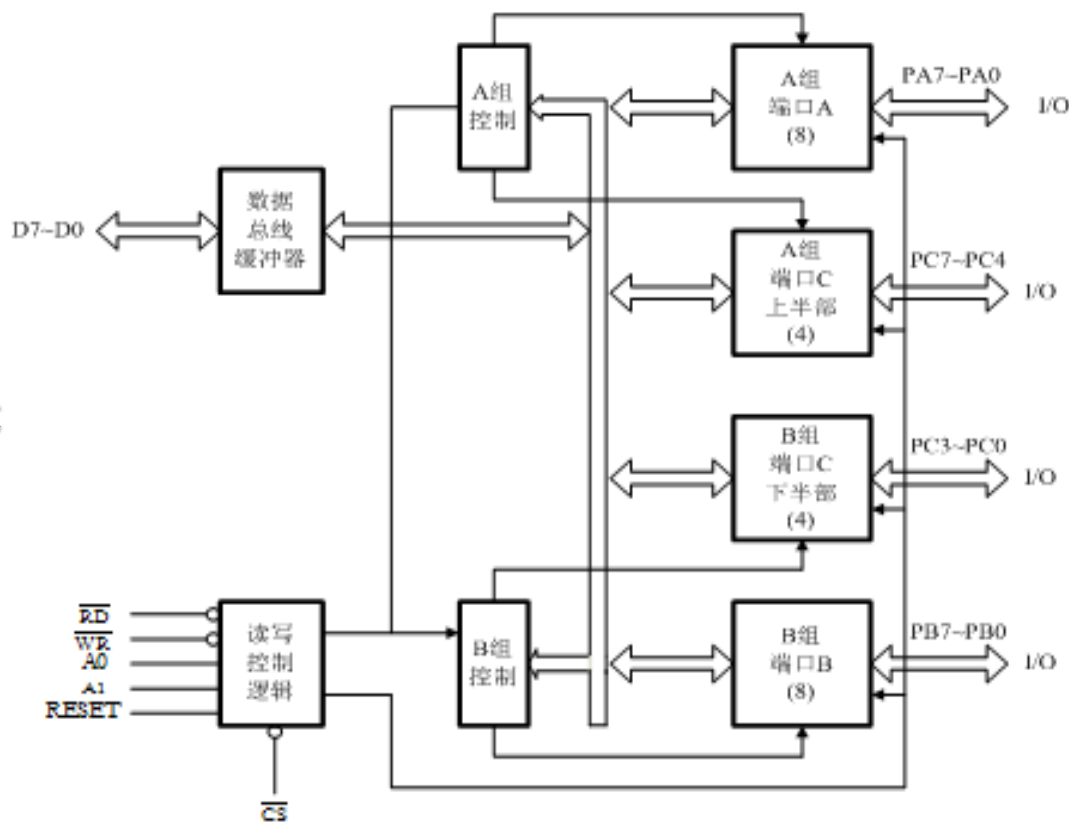


图 8-2 8255A 的内部结构

(二) 8255A引脚说明

8255A共有40只引脚，采用双列直插式封装，各引脚功能如下：

D7~D0：三态双向数据线，与单片机数据总线连接，用来传送数据信息。

$\overline{CS}$ ：片选信号线，低电平有效，表示本芯片被选中

$\overline{RD}$ ：读出信号线，控制8255A中数据的读出

$\overline{WR}$ ：写入信号线，控制向8255A数据的写入。

PA7~PA0：A口输入/输出线。

PB7~PB0：B口输入/输出线。

PC7~PC0：C口输入/输出线。

A1、A0：地址线，用来选择8255A内部的4个端口，如表8-1所示。

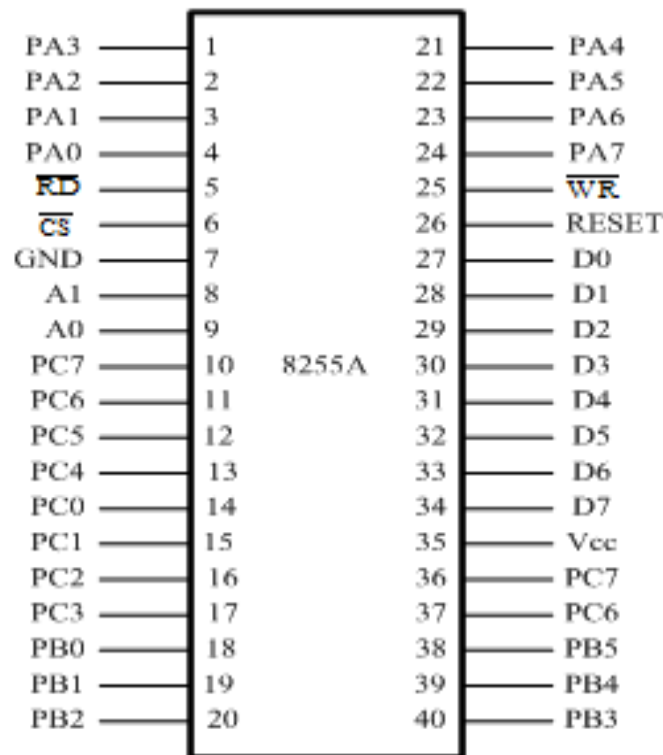


图 8-3 8255A 的引脚

8.1 I/O接口的扩展技术

表 8-1 8255A 控制信号功能表

A1	A0	$\overline{CS}$	$\overline{RD}$	$\overline{WR}$	端口	功能
0	0	0	0	1	A 口	读 A 口 (A 口数据→数据总线)
0	0	0	1	0	A 口	写 A 口 (总线数据→A 口)
0	1	0	0	1	B 口	读 B 口 (B 口数据→数据总线)
0	1	0	1	0	B 口	写 B 口 (总线数据→B 口)
1	0	0	0	1	C 口	读 C 口 (C 口数据→数据总线)
1	0	0	1	0	C 口	写 C 口 (总线数据→C 口)
1	1	0	1	0	控制器	写控制字 (总线数据→控制字寄存器)
1	1	0	0	1	×	非法状态
×	×	1	×	×	×	总线高阻 (数据总线为三态)

8.1 I/O接口的扩展技术

◆ 8255A控制字

8255A有两种控制字即**工作方式选择控制字**和**C口置位/复位控制字**。这两个控制字以D7位为标志位，若D7=1为工作方式选择控制字，若D7=0为C口置位/复位控制字。

(一)工作方式选择控制字

3种工作方式由写入控制字寄存器的方式控制字来决定。方式控制字的格式如图8-4所示。3个端口中C被分为2个部分，上半部分随A口称为A组，下半部分随B口称为B组。其中A口可工作于方式0、1和2，而B口只能工作在方式0和1。

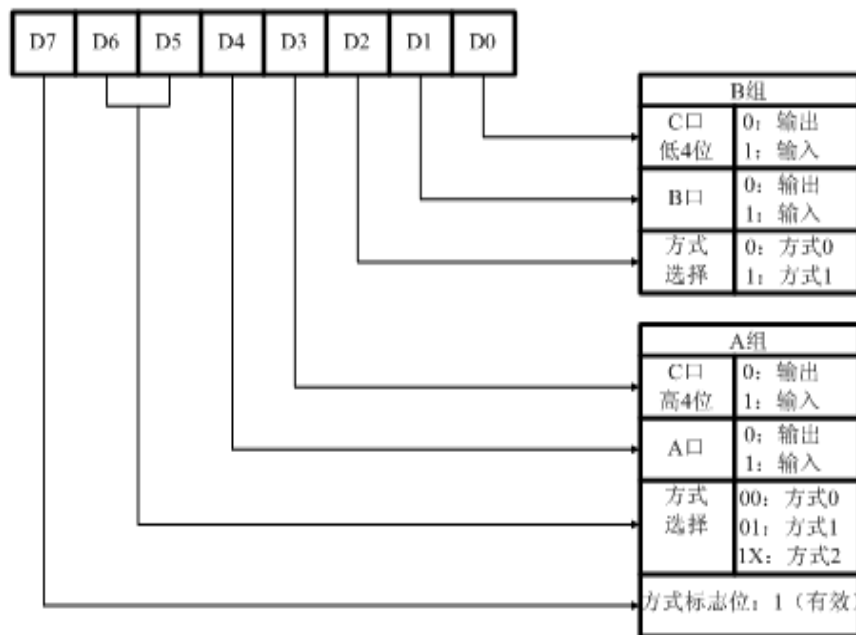


图 8-4 8255A 的方式控制字

(二) C口置位/复位控制字

本控制字可使C口8位中的任意一位置位为“1”或复位清“0”。通过控制C口的各位状态，实现某些控制功能。其控制字格式如图8-5所示。

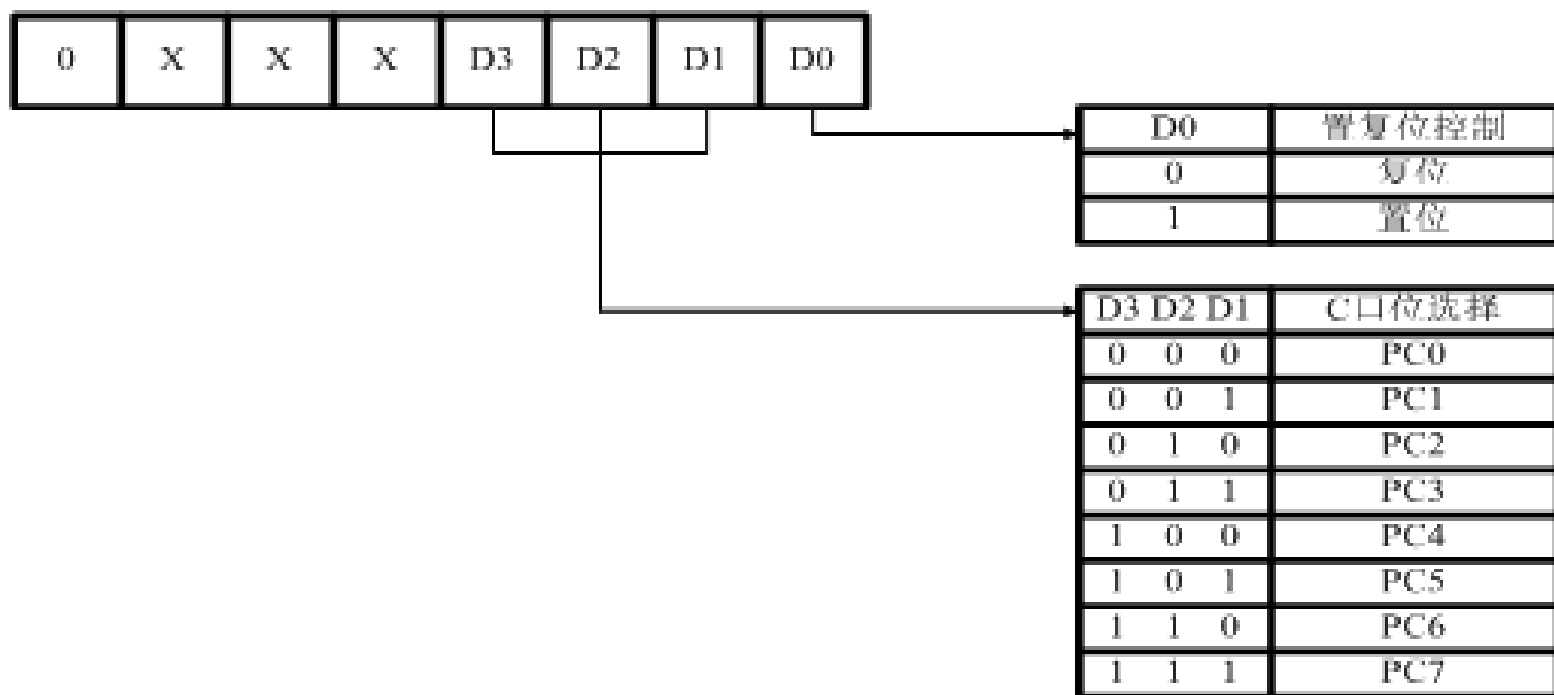


图 8-5 C口按位置位/复位控制字

8.1 I/O接口的扩展技术

◆ 8255A和AT89系列单片机的接口

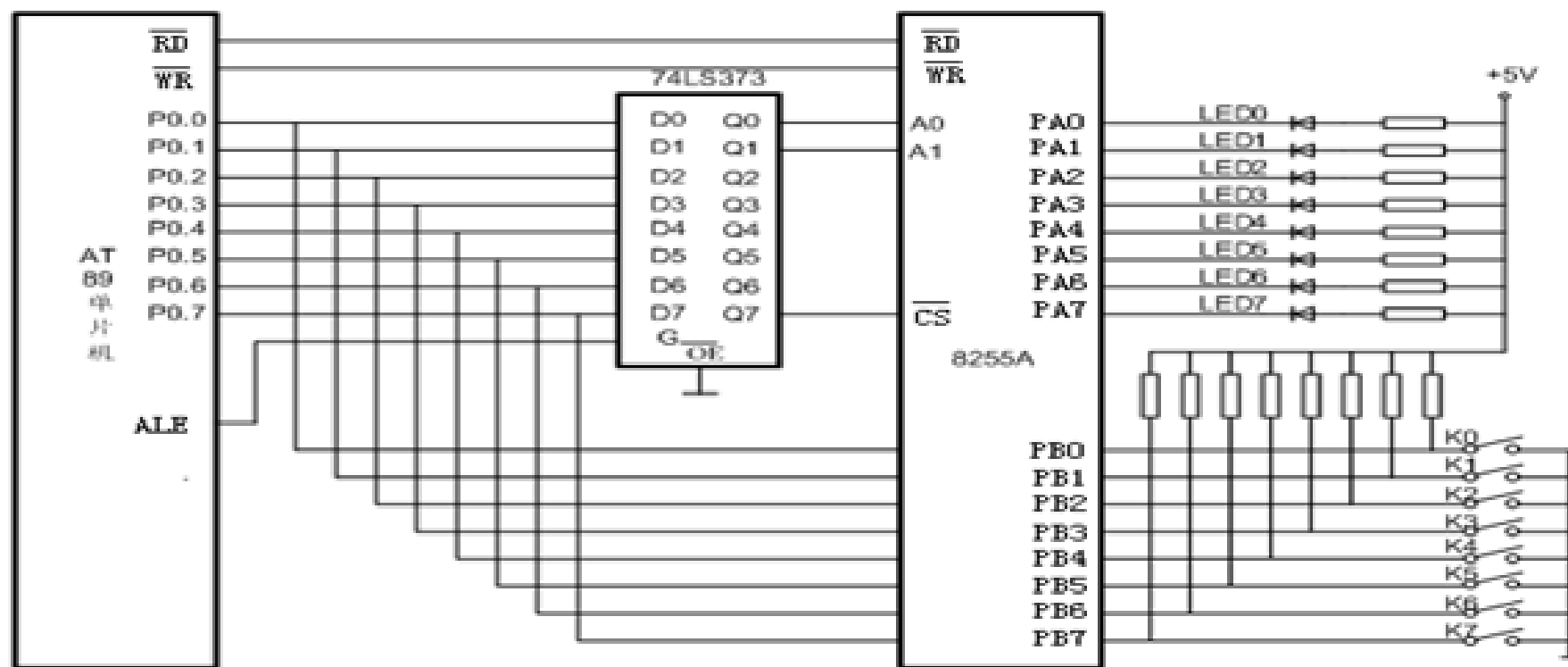


图 8-10 8255A 和 AT89 单片机的扩展电路

8.1 I/O接口的扩展技术

8255A端口地址的确定

A、B、C及控制口的地址分别为：0000H、0001H、0002H、0003H

软件编程

如图8-10所示，在8255A的B口接有8个按键，A口接有8个发光二极管，按下某键对应的发光二极管发光。实现的程序如下：

汇编语言参考程序：

```
MOV DPTR, #0FF7FH      ; 指向8255控制口
MOV A, #83H
MOVX @DPTR, A           ; 向控制口写控制字，A口输出，B口输入
LOOP: MOV DPTR, #0FF7DH ; 指向8255A的B口
MOVX @DPTR, A           ; 检测按键，将按键状态读入累加器A
MOV DPTR, #0FF7CH      ; 指向8255A的A口
MOVX @DPTR, A           ; 驱动LED发光
SJMP LOOP
```


8.1 I/O接口的扩展技术

C语言参考程序:

```
#include<reg52.h>
#include<absacc.h>
#define portA XBYTE[0xff7c]
#define portB XBYTE[0xff7d]
#define portCR XBYTE[0xff7f]
unsigned char i;
void main()
{
    while (1)
        portCR=0x83;    // 向8255A控制口写入控制字，A口输出，B口输入
    while(1)
    {
        i=portB;    //监测8255A的B口按键，读入按键状态
        portA= i;    // 向8255A的A口输出数据，驱动LED发光
    }
}
```

8.2 键盘及其与单片机的接口技术

◇ 键盘的工作原理

键盘是由一组规则排列的按键组成，一个按键实际上是一个开关元件，也就是说，键盘是一组规则排列的开关。

◆ 按键的分类

结构原理 { 触点式开关按键
无触点式开关按键

接口原理 { 编码键盘
非编码键盘

8.2 键盘及其与单片机的接口技术

◆ 按键的结构特点

微机键盘通常使用机械触点式按键开关，其主要功能是把机械上的通断转换成为电气上的逻辑关系。能提供标准的TTL逻辑电平，以便与通用数字系统的逻辑电平相容。

机械式按键在按下或释放时，由于机械弹性作用的影响，通常伴随有一定时间的触点机械抖动，抖动时间的长短与开关的机械特性有关，一般为5-10ms

在触点抖动期间检测按键的通与断状态，可能导致判断出错，必须采取去抖动措施，可采用硬件去抖与软件去抖的方法。

8.2 键盘及其与单片机的接口技术

(一) 硬件去抖动

可采用在键输出端加R—S触发器或单稳态触发器构成去抖动电路。

图8-12是一种由R-S触发器构成的去抖动电路。

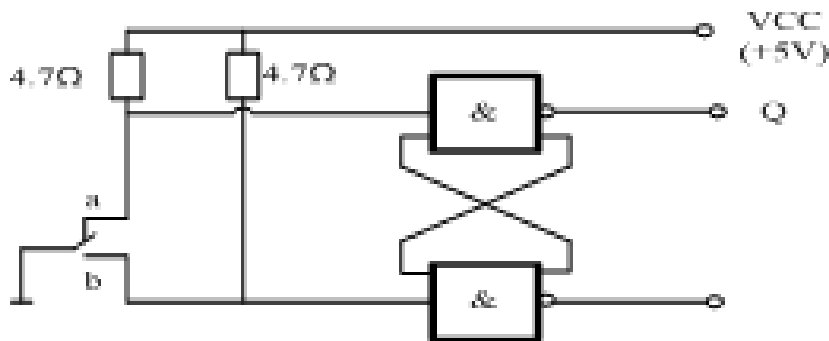


图 8-12 双稳态去抖动电路

(二) 软件去抖动

在检测到有按键按下时，执行一个10ms左右的延时程序后，再确认该按键电平是否仍保持闭合状态电平，若仍保持闭合状态电平，则确认该键处于闭合状态。

8.2 键盘及其与单片机的接口技术

◆ 按键编码

一组按键或键盘都要通过接口线查询按键的开关状态。根据键盘结构的不同，采用不同的编码。无论有无编码，以及采用什么编码，最后都要转换为相对应的键值，以实现按键功能程序的跳转。

◆ 编制键盘程序

一个完善的键盘控制程序应具备以下功能：

- (1) 检测有无按键按下，并采取硬件或软件措施，消除键盘按键机械触点抖动的影响。
- (2) 判断是哪一个键按下，并且每次只处理一个按键。
- (3) 准确输出按键值(或键号)，以满足跳转指令要求。

8.2 键盘及其与单片机的接口技术

◇独立式按键与单片机的接口

键盘是由一组规则排列的按键组成，一个按键实际上是一个开关元件，也就是说，键盘是一组规则排列的开关。

◆独立式按键结构

独立式键盘的结构如图8-13所示，这是最简单的键盘结构形式。

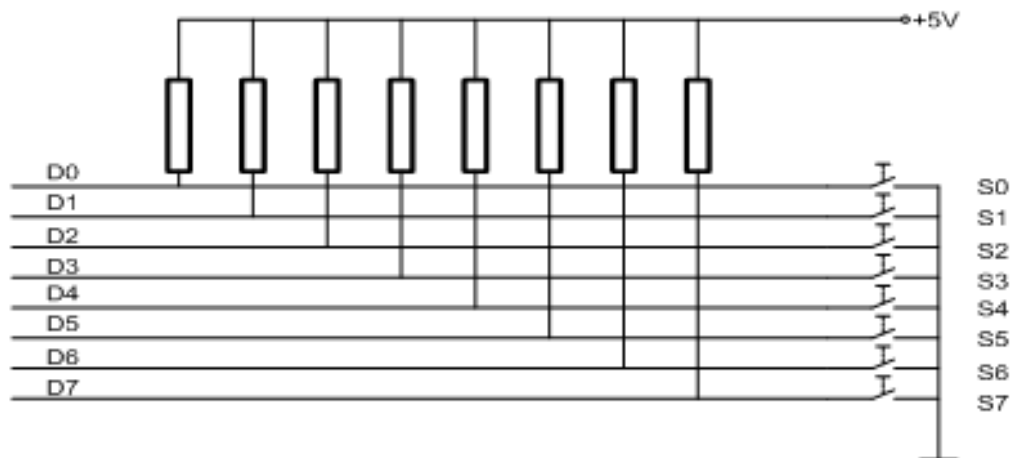
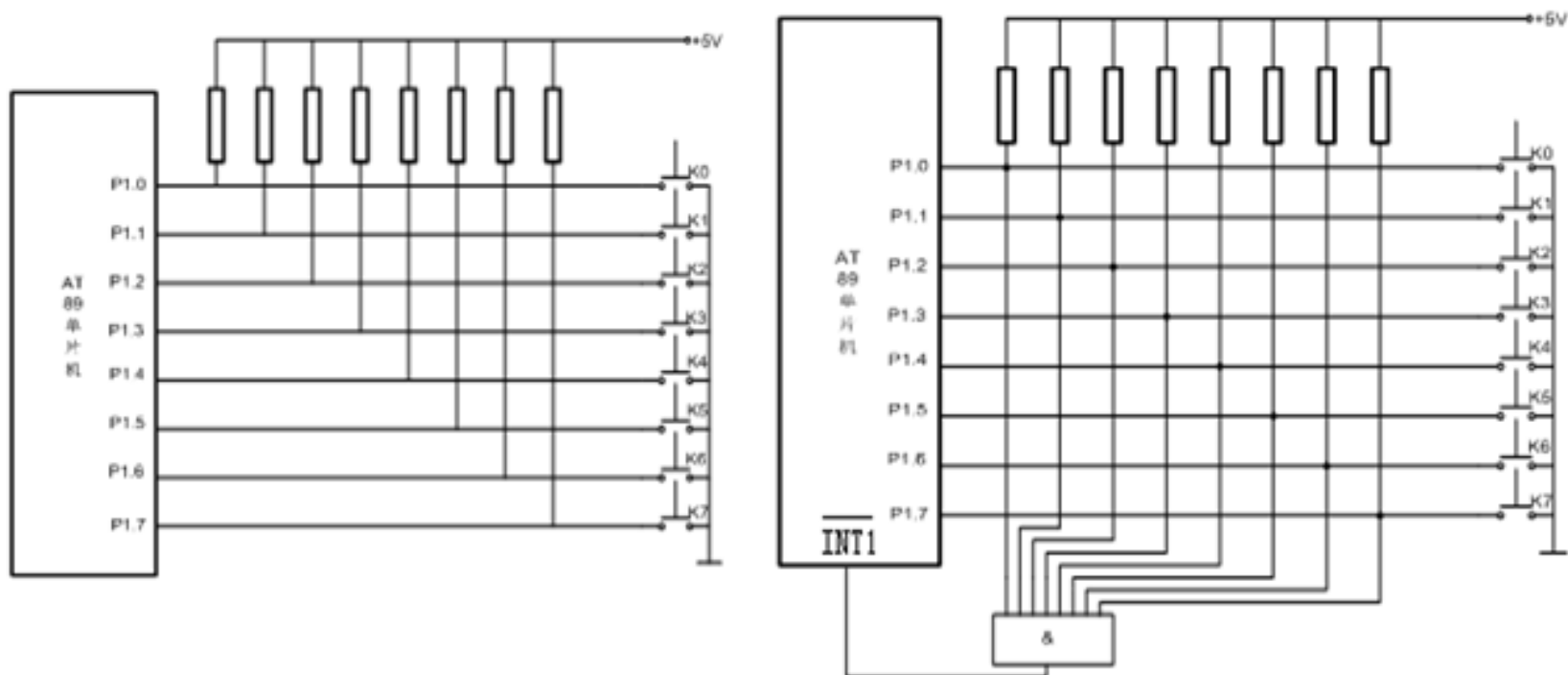


图 8-13 独立式键盘结构

8.2 键盘及其与单片机的接口技术

◆独立式键盘与AT89单片机的接口

图8-14为独立式按键程序查询方式和中断方式的接口电路。



a) 程序查询方式按键接口电路

b) 中断方式按键接口电路

图 8-14 独立式按键接口电路

8.2 键盘及其与单片机的接口技术

◆独立式按键的软件设计

程序清单如下：

```
START:  MOV A, #0FFH          ; 置P1口为输入方式
        MOV P1, A
LOOP:   MOV A, P1              ; 读键值
        CJNE A, #0FFH, L0      ; 有键按下否
        SJMP LOOP              ; 无键按下等待
L0:     LCALL DELAY             ; 调延时去抖
        MOV A, P1              ; 重新读入键盘状态
        CJNE A, #0FFH, L1      ; 非误读, 转
        SJMP LOOP
L1:     JNB ACC.0, K0           ; 为0号键按下, 转相应程序K0
        JNB ACC.1, K1         ; 为1号键按下, 转相应程序K1
        JNB ACC.2, K2
        JNB ACC.3, K3
```




JNB	ACC. 5,	K5	
JNB	ACC. 6,	K6	
JNB	ACC. 7,	K7	
SJMP	START		; 无键按下, 返回
K0:	AJMP	KEY0	; 各功能键程序入口地址表
K1:	AJMP	KEY1	
K2:	AJMP	KEY2	
K3:	AJMP	KEY1	
K4:	AJMP	KEY2	
	K5:	AJMP	KEY1
K6:	AJMP	KEY2	
K7:	AJMP	KEY1	
KEY0:	...		; 0号功能键处理程序
	LJMP	START	; 0号键处理程序执行完返回
	...		
KEY7:	...		; 7号功能键处理程序
	LJMP	START	; 7号键处理程序执行完返回

◇ 矩阵式键盘与单片机的接口

◆ 矩阵式键盘的结构及工作原理

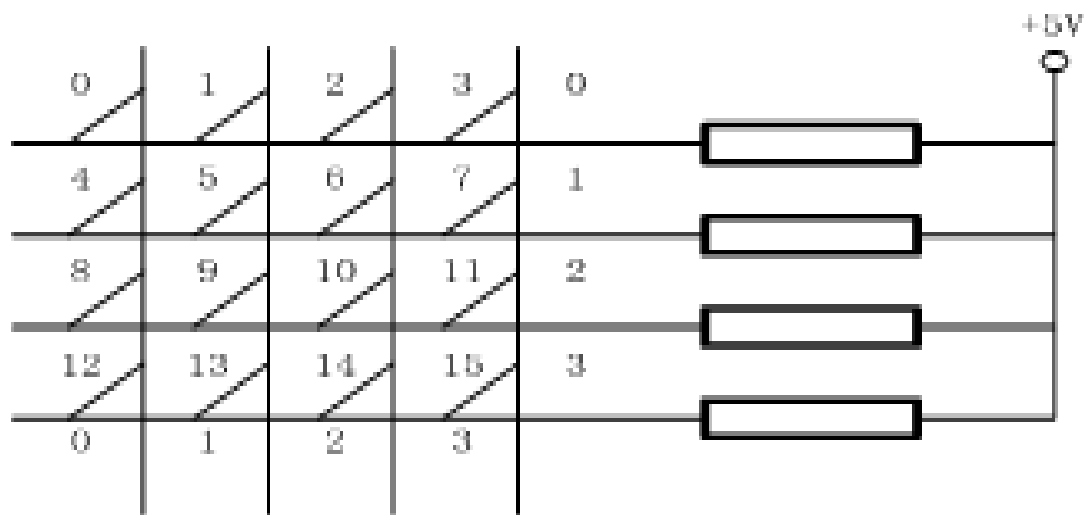


图 8-15 矩阵式键盘结构

◆键盘的编码

对于矩阵式键盘，按键的位置由行号和列号唯一确定，因此可分别对行号和列号进行二进制编码，然后将两值合成一个字节，高4位是行号，低4位是列号。

◆矩阵式键盘按键的识别

扫描法：

- 1) 判断有无键按下。
- 2) 如果有键按下，识别是哪一个键按下，键盘扫描取得闭合键的行、列值。
- 3) 用算法或查表法得到键值。
- 4) 判断闭合键是否释放，如没释放则继续等待。
- 5) 将闭合键键号保存，同时转去执行该闭合键的功能。

8.2 键盘及其与单片机的接口技术

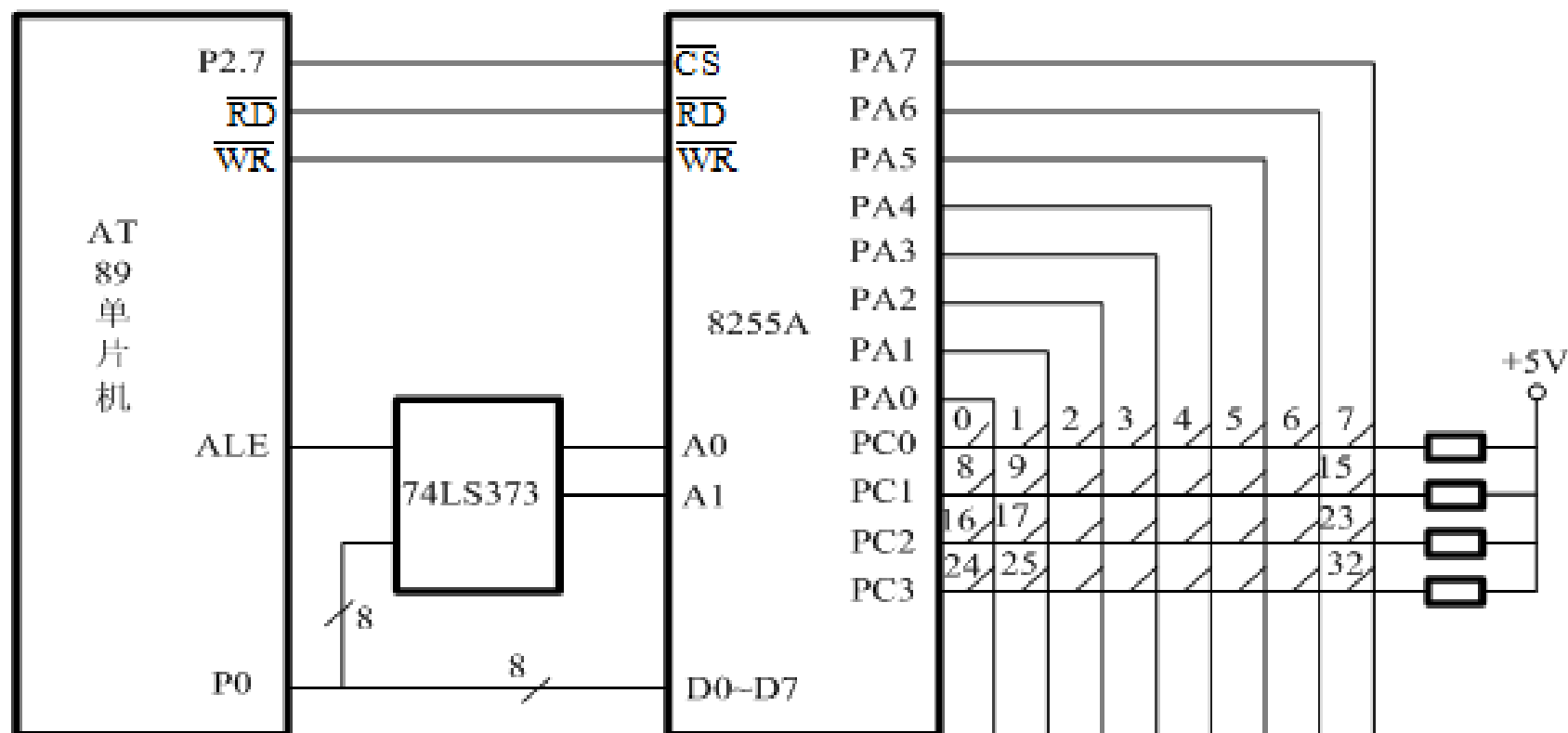


图 8-16 采用扩展 8255A 组成的 4×8 矩阵式键盘

8.2 键盘及其与单片机的接口技术

◆ 键盘的工作方式

(1) 编程扫描方式

利用**CPU**在完成其它工作的空余时间，调用键盘扫描子程序来响应键盘输入的要求。

(2) 定时扫描工作方式

每隔一定的时间对键盘扫描一次，它利用单片机内部的定时器产生一定时间（例如**10ms**）的定时，当定时时间到就产生定时器溢出中断，**CPU**响应定时器溢出中断请求，对键盘进行扫描

(3) 中断扫描工作方式

当无键按下时，**CPU**处理自己的工作，不需要理睬键盘；当有键按下时，产生中断请求，**CPU**转去执行键盘扫描子程序，并识别键号。

◆ 矩阵式键盘的软件设计

键盘的硬件电路如图8-17所示。

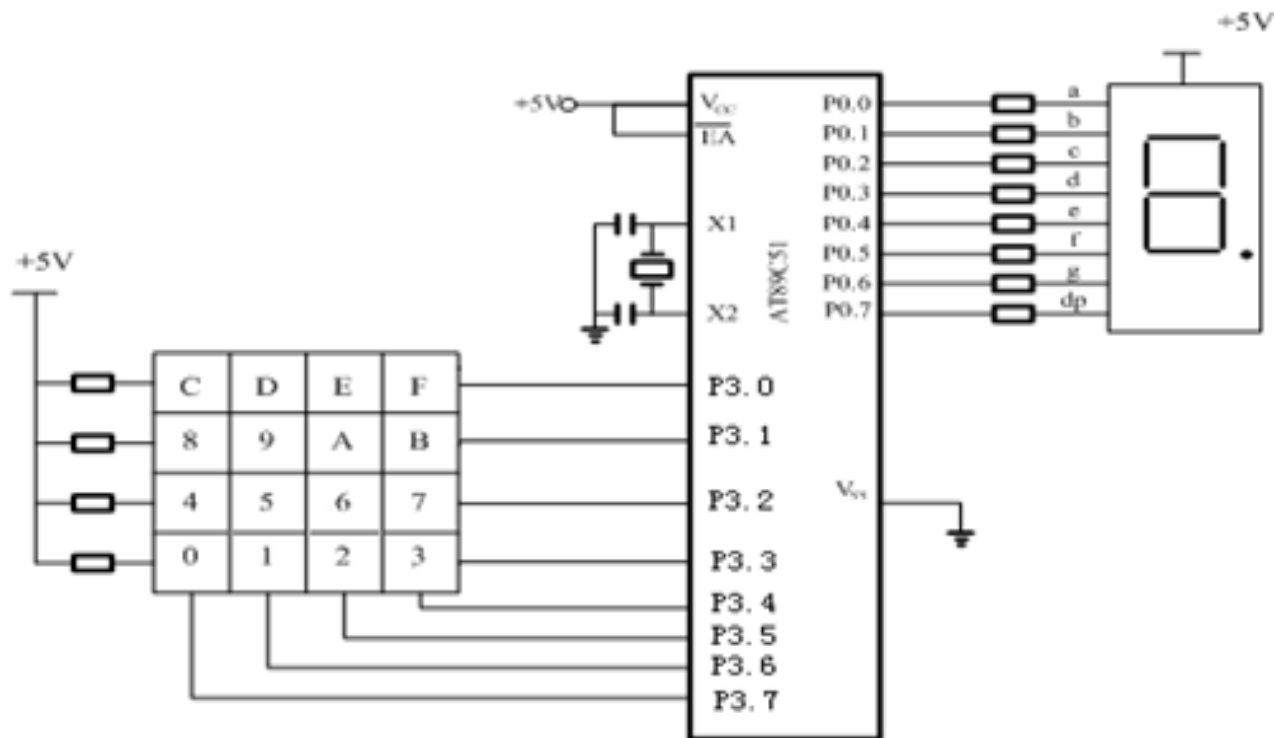


图 8-17 4×4 矩阵式键盘电路

扫描时，首先将行设置为低电平，在判断有键按下后，读入列状态。如果列状态出现并非全部为1状态，这时0状态的列与行相交的键就是被按下的键。

C语言程序:

```
#include<reg51.h>
#include<intrins.h>
#include<absacc.h>
#define uchar unsigned char
#define uint unsigned int
uchar code a[16]={0xc0H, 0xf9H,
0xa4H, 0xb0H, 0x99H, 0x92H,
0x82H, 0xf8H, 0x80H, 0x90H,
0x88H, 0x83H,0xc6H, 0xa1H,
0x86H, 0x8eH};//字型码表
```

```
void delay(uint i)    //延时程序
{
    uint j;
    for(j=0;j<i;j++);
}
```

```
uchar checkkey( )    //检测有没有键按下,无
键按下返回0
{
    uchar i ;
    uchar j ;
    j=0x0f;
    P3=j;             // 列输出全0
    i=P3;             //读行线状态
    i=i&0x0f;
    if (i==0x0f) return (0); //无键按下返回0
    else return (0xff);
}
```

```
uchar keyscan( )
```

```
//键盘扫描程序，有键按下返回键码；无键按下返回0xff.
```

```
{
```

```
    uchar scancode;
```

```
    uchar codevalue;
```

```
    uchar a;
```

```
    uchar m=0;
```

```
    uchar k;
```

```
    uchar i, j;
```

```
    if (checkkey()==0) return (0xff);
```

```
    else
```

```
    {delay(100);
```

```
        if (checkkey()==0) return (0xff);
```

```
        else
```

```
        {
```

```
            scancode=0xf7;m=0x00;
```

```
//键盘行扫描初值，m为首行键号
```

```
            for (i=1;i<=4;i++)
```

```
//i为行号，4行逐行扫描
```

```
            {
```

```
                k=0x80;
```

```
//列检测码
```

```
                P3=scancode;
```

```
//从P3.3开始，逐行扫描
```

```
                a=P3;
```

```
                for (j=0;j<4;j++)
```

```
//j为列号，检测本行的4列是否有键按下
```

```
                {
```

```
                    if ((a&k)==0)
```

```
//检测本列是否有键按下
```

```
                    {
```

```
                        codevalue = m+j;
```

```
//键值=首行键号+列号
```

```
                        while (checkkey()!=0);
```

```
//判断按键是否释放
```

```
                        return (codevalue);
```

```
//释放则返回按键值
```

```
                    }
```

```
                    else k=k>>1;
```

```
//本列无键按下，列检测码右移
```

```
                }
```

```
            m=m+4;
```

```
//本行无键按下，行首键号+4
```

```
            scancode= ~scancode;
```

```
//行扫描码右移，为scancode右移时，移入的数为1。
```

```
            scancode=scancode>>1;
```

```
            scancode=~scancode;
```

```
        }
```

```
    }
```

```
}
```




```
void main()
```

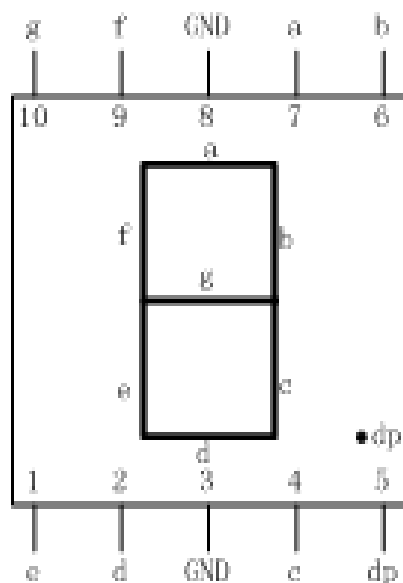
```
//主函数
```

```
{  
    int x;  
    P0=0xff;  
    while(1)  
    {  
        if (checkkey()==0x00) continue;  
        else  
        {  
            x= keyscan();  
            P0=a[x];    //显示按键值  
            delay(100);  
        }  
    }  
}
```

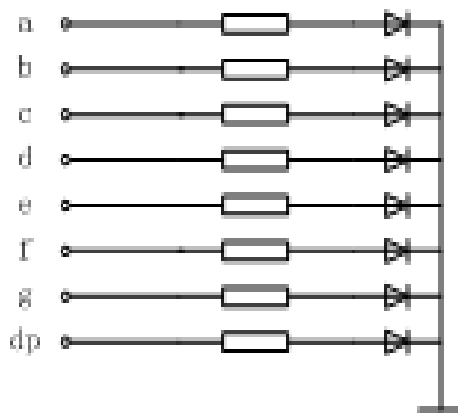
8.3 LED显示器及其与单片机的接口技术

◇LED显示器的结构与原理

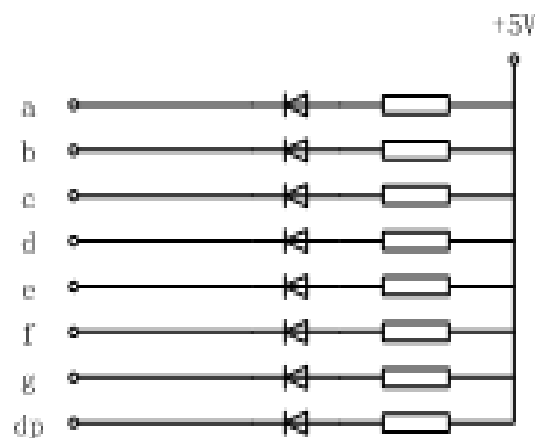
◆LED显示器结构



(a) 管脚配置



b) 共阴极



(c) 共阳极

图 8-18 LED 显示器原理图

8.3 LED显示器及其与单片机的接口技术

◆LED显示器工作原理

为了使LED显示器显示不同的符号或数字，就要把不同段的发光二极管点亮，这样就需要为LED显示器提供代码，因为这些代码是为了显示字型的，因此称之为字型码（各段与字节中各位对应关系如表8-2所示。或段码）。7段发光二极管和1个小数点位，共计8段。

表 8-2 LED 各段与字节中各位对应关系

代码位	D7	D6	D5	D4	D3	D2	D1	D0
显示段	dp	e	f	e	d	c	b	a

表 8-3 8 段 LED 段码

显示字符	共阴极段码	共阳极段码	显示字符	共阴极段码	共阳极段码
0	3FH	COH	C	39H	C6H
1	06H	F9H	D	5EH	A1H
2	5BH	A4H	E	79H	86H
3	4FH	BOH	F	71H	8EH
4	66H	99H	P	73H	8CH
5	6DH	92H	U	3EH	C1H
6	7DH	82H	T	31H	CEH
7	07H	F8H	y	6EH	91H
8	7FH	80H	H	76H	89H
9	6FH	90H	L	38H	C7H
A	77H	88H	“灭”	00H	FFH
B	7CH	83H	...	...	...

8.3 LED显示器及其与单片机的接口技术

◇ LED显示器的译码方式

译码方式是指由显示字符转换得到对应的字段码的方式分为硬件译码和软件译码两种方式。硬件译码方式是指利用专门的硬件电路来实现显示字符到字段码的转换，软件译码方式就是通过编写软件译码程序，通过译码程序来得到要显示的字符的字段码。

8.3 LED显示器及其与单片机的接口技术

◇ LED显示器的显示方式

LED显示器有静态和动态2种显示方式。

◆ LED静态显示方式

LED静态显示时，其公共端直接接地（共阴极）或接电源（共阳极），各段选线分别与I/O口线相连。要显示字符，直接在I/O线送相应的字段码，如图8-19所示。

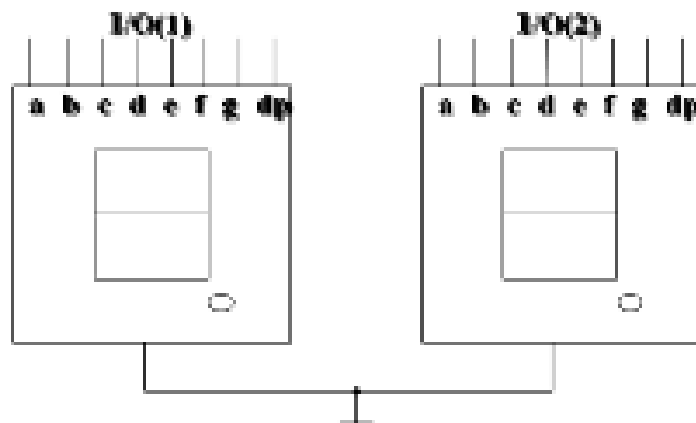


图 8-19 静态显示

◆LED动态显示方式

LED动态显示是将所有的数码管的段选线并接在一起，用一个**I/O口**控制，公共端不是直接接地（共阴极）或电源（共阳极），而是通过相应的**I/O口线**控制，如图8-20所示。

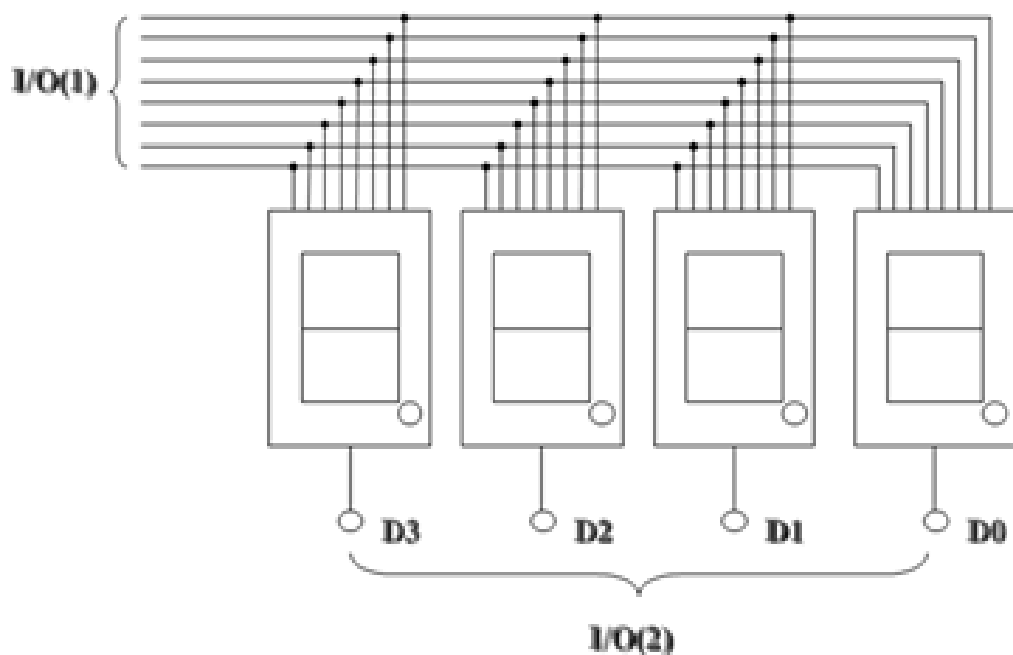


图 8-20 动态显示

8.3 LED显示器及其与单片机的接口技术

◇ LED显示器与单片机的接口

◆ 单片机与LED显示器静态显示接口

硬件电路连接如图8-21所示，单片机P0端口接有一只LED显示器

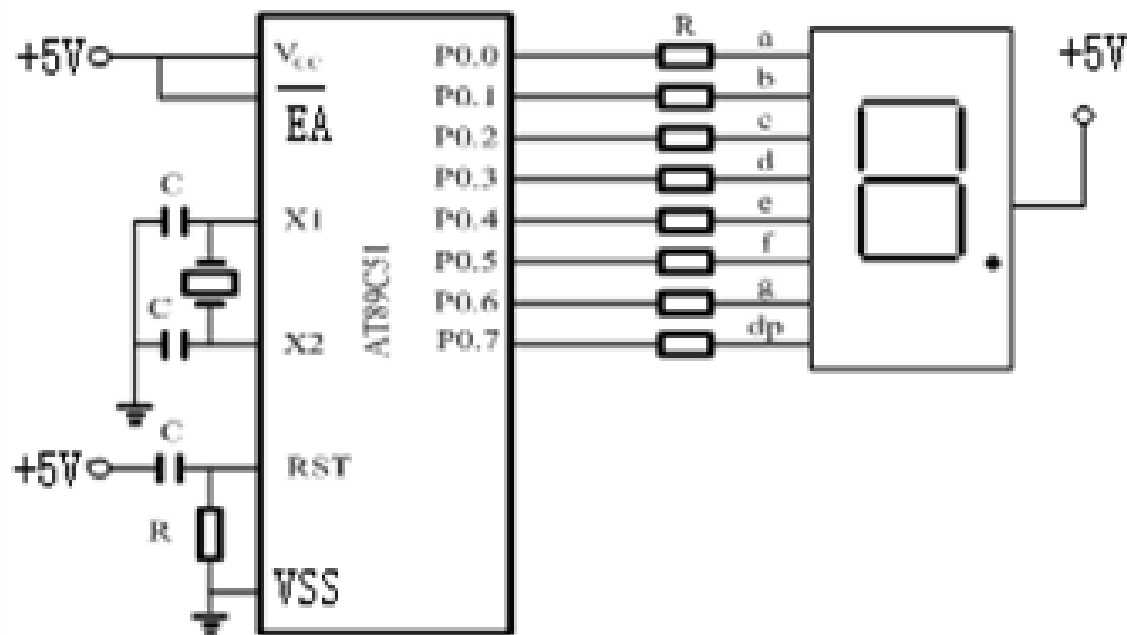


图 8-21 LED 静态显示接口电路

8.3 LED显示器及其与单片机的接口技术

◆单片机与LED显示器静态显示接口

软件设计：

硬件电路连接如图8-21所示，在显示器上显示数字“7”的汇编语言和C语言程序参考课本。



汇编语言:

```
START:  MOV DPTR, #TABLE    ; 存入表的起始地址
        MOV A, #7           ; 将欲显示的数字7存入A
        MOVC A, @A+DPTR     ; 按地址取代码并存入A
        MOV P0, A           ; 将代码送P0转变为数字显示
        SJMP $              ; 程序运行在当前状态

TABALE:  DB 0C0H, 0F9H, 0A4H, 0B0H, 99H, 92H, 82H, 0F8H,
          80H, 90H ; 0-9字型码表

        END
```

C语言源程序代码:

```
unsigned char table[]={0xc0H, 0xf9H, 0xa4H, 0xb0H, 0x99H, 0x92H,
0x82H, 0xf8H, 0x80H, 0x90H };    //字型码表

void main()
{
    P0=table(7);    // 显示数字 “7”
    While(1);
}
```

◇ LED显示器与单片机的接口

◆ 单片机与LED显示器动态显示接口

硬件电路连接如图8-22所示。

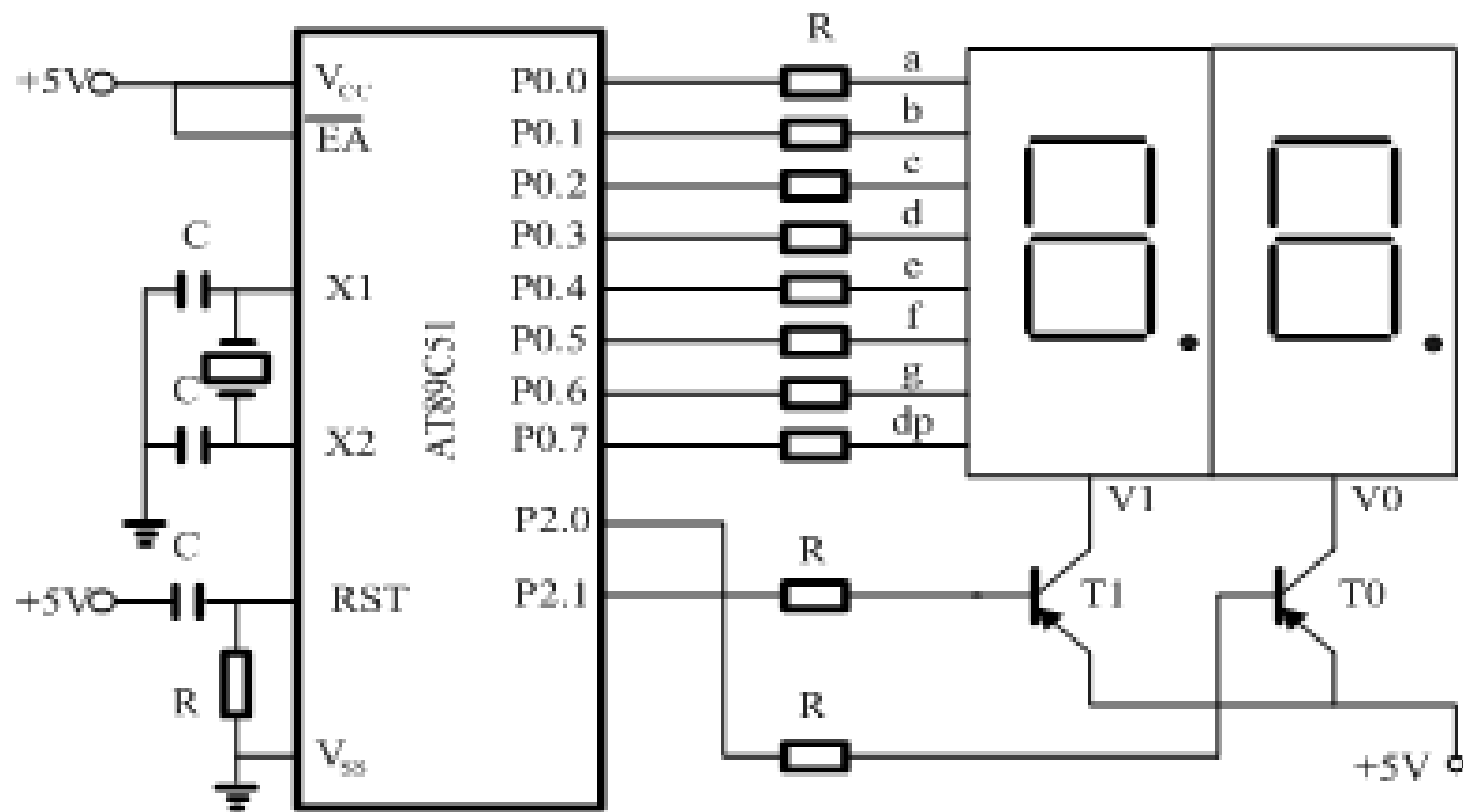


图 8-22 LED 动态显示接口电路

8.3 LED显示器及其与单片机的接口技术

◆单片机与LED显示器动态显示接口

软件设计:

硬件电路连接如图8-22所示。单片机P0口和LED显示器8段相连，P2口的P2.0和P2.1通过三极管T0和T1连接LED显示器的公共端，控制与其相连的数码管显示器工作。将R0内容(0-99)循环显示在两个LED显示器的汇编语言和C语言程序参考课本。

将R0内
容(0-99)
循环显
示在两
个LED
显示器的程序:

```
START:  MOV  R0, #0          ; 初始化计数器
        MOV  DPTR, #TABLE    ; 存入查表起始地址
LOOP:   MOV  A, R0
        MOV  B, #10          ; 16进制转换成10进制
        DIV  AB              ; A/B的商存入A, 余数存入B
        MOV  R1, A           ; R1存放十位数
        MOV  R2, B           ; R2存放个位数
        MOV  R3, #50         ; 设导通频率为50
LOOP1:  MOV  A, R2           ; 个位数显示
        ACALL CHANG          ; 调用显示子程序
        CLR  P2.0            ; 开个位显示
        ACALL DLY10ms        ; 调用延时10ms子程序
        SETB P2.0           ; 关闭个位显示
        MOV  A, R1           ; 取十位数
        ACALL CHANG          ; 调用显示子程序
        CLR  P2.1            ; 开十位显示
        ACALL DLY10ms        ; 调用延时10ms子程序
        SETB P2.1           ; 关闭十位显示
        DJNZ R3, LOOP1       ; 50次显示未完成继续扫描
        INC  R0              ; 计数器加1
        CJNE R0, #100, LOOP  ; 没到100
        SJMP START          ; 跳转到开始处
CHANG:  MOVC  A, @A+DPTR     ; 显示子程序
        MOV  P0, A
        RET
DLY10ms: MOV  R6, #20        ; 10ms延时子程序
        D1:  MOV  R7, #248
        DJNZ R7, $
        DJNZ R6, D1
        RET                  ; 延时子程序返回
TABLE:  DB  0C0H, 0F9H, 0A4H, 0B0H, 99H, 92H, 82H, 0F8H, 80H, 90H
        END                  ; 程序结束
```

C语言程序:

```
#include<reg52.h>

unsigned char code table[]={0xc0,0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80,0x90};

unsigned char dispcount=0;    //计数

sbit gewei=P2^0;              //各位选通定义
sbit shiwei=P2^1;            //十位选通定义

void Delay(unsigned int tc)  //延时程序
{
    while(tc!=0)
    {
        unsigned int i;
        for(i=0;i<100;i++);
        tc--;
    }
}
```



```
void LED(unsigned char x )           //led显示函数
{
    if(x>=10)                        //显示两位数
    {
        shiwei=0;
        P0=table[x/10];              //显示十位数
        Delay(10);
        shiwei=1;
        gewei=0;
        P0=table[x%10];              // 显示个位数
        Delay(10);
        gewei=1;
    }
    else                             //显示一位数
    {
        shiwei=1;
        gewei=0;
        P0=table[x];
        Delay(10);
    }
}
```



```
void main()
{
    while(1)          //循环执行
    {
        LED(dispcount);  //调用显示函数
        dispcount++;     //计数器加1
        if(dispcount==100); //达到99后重新开始
        dispcount=0;
    }
}
```


8.4 LCD显示器及其接口技术

◇LCD显示器的结构与原理

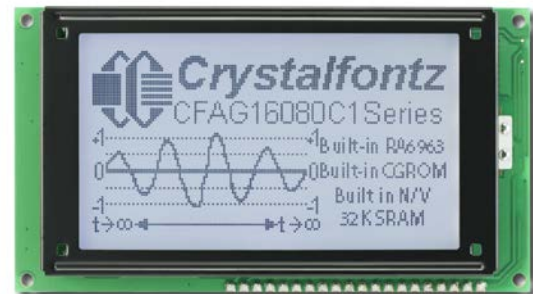
LCD (Liquid Crystal Display) 是液晶显示器英文名称的缩写，液晶显示器是一种被动式的显示器，即液晶本身不发光，而是利用液晶经过处理后能改变光线通过方向的特性，达到白底黑字或黑底白字显示的目的。

◆LCD显示器的分类

LCD显示器分为：

- ① 段式
- ② 点阵式

点阵式又可分为
字符模式LCD和
图形模式LCD



8.4 LCD显示器及其接口技术

◆LCD模块的引脚

下面介绍常用的20字×2行字符模块，引脚如图8-23所示。

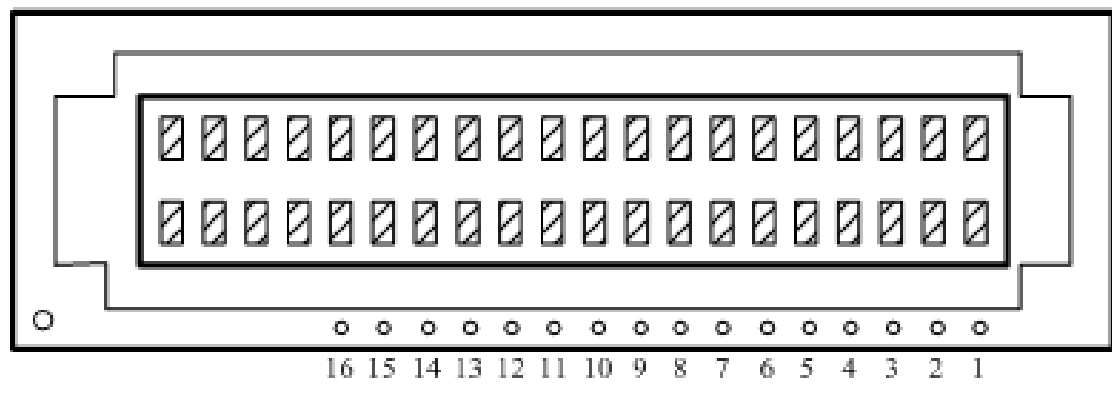
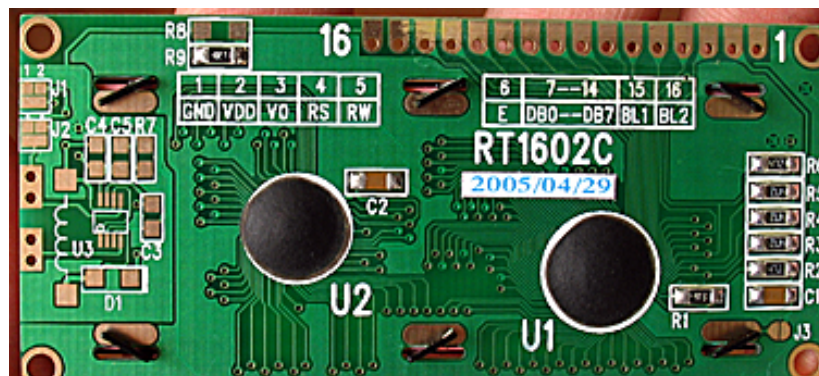
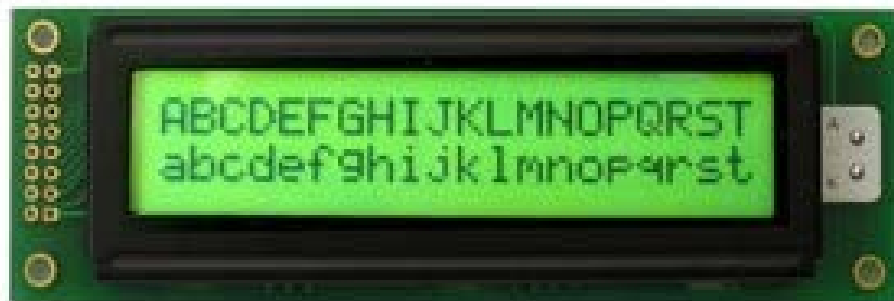


图 8-23 LCD 引脚图

8.4 LCD显示器及其接口技术

表 8-4 LCD 引脚功能

引脚	符号	功能说明
1	VSS	接地
2	VDD	电源 +5V
3	VO	显示屏明亮度调整脚，一般将此脚接地
4	RS	寄存器选择 0：命令寄存器写入，忙标志和地址计数器读出； 1：数据寄存器写入、读出
5	$R/\overline{W}$	读/写 0：写；1：读
6	E	读/写使能，下降沿触发
7	DB0	8 位双向数据总线，实现数据的传输；DB7 也是一个忙标志 (Busy flag)
8	DB1	
9	DB2	
10	DB3	
11	DB4	
12	DB5	
13	DB6	
14	DB7	
15	BLA	背光源正极
16	BLK	背光源负极

8.4 LCD显示器及其接口技术

◆寄存器选择及显示器地址

(一) 寄存器选择

LCD内部有两个寄存器，一个指令寄存器**IR**，另一个是数据寄存器**DR**。**IR**用来存放由微控制器所送来的指令代码，如清除显示、光标归位等，**DR**用来存放欲显示的数据。

显示的过程是先把欲存放数据地址写入**IR**，再把欲显示的数据写入**DR**，**DR**自动把数据送至相应的**DD RAM**或**CG RAM**，**DDRAM**是显示数据的存储器，用来存放**LCD**的显示数据；**CG RAM**是字符产生器，用来存放设计的**5×7**点图形的显示数据。

8.4 LCD显示器及其接口技术

LCD指令寄存器和数据寄存器的选择如表8-5 所示

表 8-5 LCD 寄存器的选择

E	R / $\overline{W}$	RS	功能说明
1	0	0	命令寄存器写入
1	0	1	数据寄存器写入
1	1	0	忙标志和地址计数器读出
1	1	1	数据寄存器读出
0	X	X	不动作

(二)显示器地址

20字×2行显示器地址如表8-6所示。

表 8-6 20×2 LCD 显示器地址

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F	90	91	92	93
C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF	D0	D1	D2	D3

8.4 LCD显示器及其接口技术

◇ AT89单片机与LCD模块的接口

◆ 硬件连接

LCD模块与单片机连接电路非常简单，如图8-24所示。

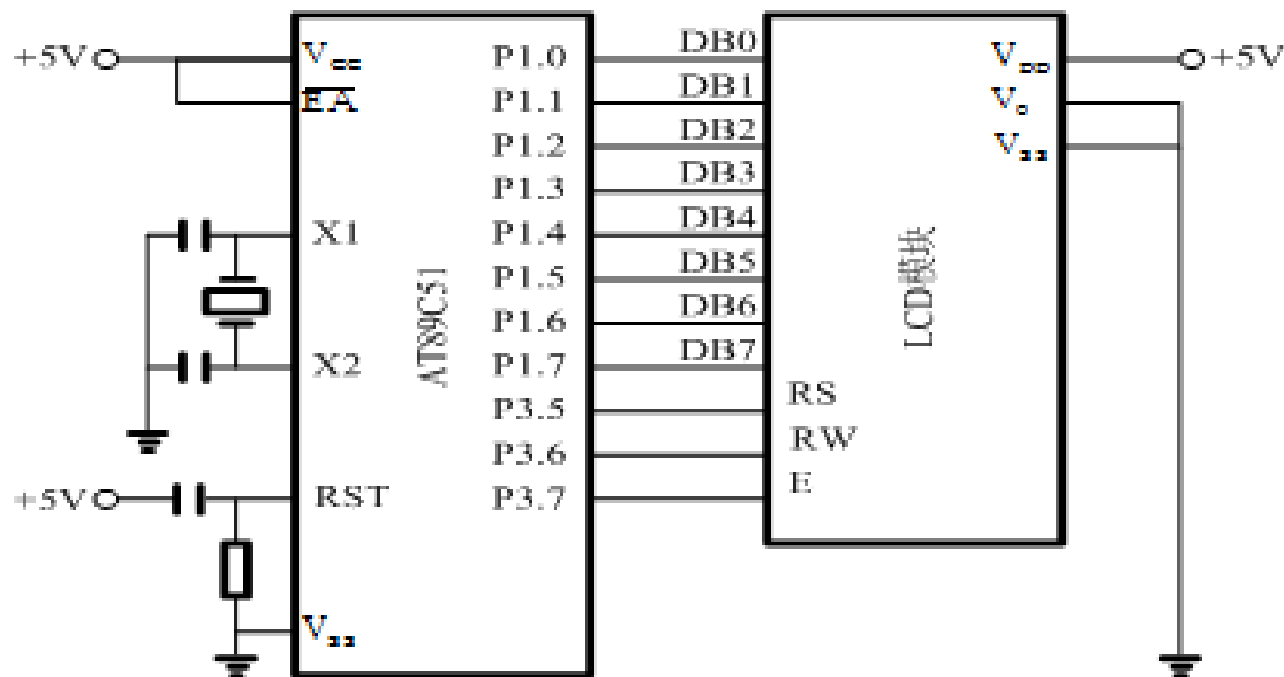


图 8-24 LCD 与单片机连接图

8.4 LCD显示器及其接口技术

◇ AT89单片机与LCD模块的接口

◆ 软件设计

根据以上硬件连接电路图进行软件设计，在LCD显示器上形式” OK”的程序见课本（略）



根据以上硬件连接电路图，在LCD显示器上形式” OK”的程序如下：

RS bit P3.5

RW bit P3.6

E bit P3.7

LCD EQU P1

； 主程序

MAIN:MOV LCD, #01H	;清屏并复位光标
ACALL WR_COMN	;调用写指令子程序
ACLL INIT_LCD	;调用初始化子程序
MOV LCD, #82H	;写入显示起始地址
ACALL WR_COMN	;调用写指令子程序
MOV LCD, #'O'	;显示 “O”
ACALL WR_DATA	;调用写数据子程序
MOV LCD, #'K'	;显示 “K”
ACALL WR_DATA	;调用写数据子程序
SJMP \$	

； LCM初始化子程序

INTI_LCD: MOV A, #38H	;设置8行、2行、5×7点阵
ACALL WR_COMN	;调用写指令子程序
MOV A, #0FH	;显示器开，光标允许闪烁
ACALL WR_COMN	;调用写指令子程序
MOV A, #06H	;文字不动，光标自动右移
ACALL WR_COMN	;调用写指令子程序
RET	;子程序返回

； 写指令子程序

WR_COMN: ACALL CHECK_BF	;调用判断LCM忙碌子程序
MOV LCD,A	
CLR RS	;RS=0, 选择指令寄存器
CLR RW	;RW=0, 选择写模式
CLR E	
NOP	
SETB E	
RET	;子程序返回

； 判断忙子程序

CHECK_BF:MOV LCD, #0FEH	;此时不接收外来指令
CLR RS	;RS=0, 选择指令寄存器
SETB RW	;RW=1, 选择读模式
CLR E	;E=0, 禁止读/写LCM
NOP	;延时1us
SETB E	;E=1, 允许读/写LCM
JB LCD.7, CHECK_BF	;忙碌循环等待
RET	;子程序返回

； 写数据子程序

WR_DATA: ACALL CHECK_BF	;调用判断LCM忙碌子程序
MOV LCD,A	
SETB RS	;RS=1, 选择数据寄存器
CLR RW	;RW=0, 选择写模式
CLR E	
NOP	
SETB E	
RET	;子程序返回
END	;程序结束

C语言程序:

```
#include <reg51.h>

#define uchar unsigned char
#define uint unsigned int

sbit lcden=P3^7;
sbit lcdrw=P3^6;
sbit lcdrs=P3^5;

char num;

void delay(uint z)           // 延时子程序
{
    uint x,y;
    for(x=z;x>0;x--)
        for(y=110;y>0;y--);
}
```



```
void write_com(uchar com)           // 写指令子程序
```

```
{  
    lcdrs=0;  
    P1=com;  
    delay(5);  
    lcden=1;  
    delay(5);  
    lcden=0;  
}
```

```
void write_data(uchar date)        // 写数据子程序
```

```
{  
    lcdrs=1;  
    P1=date;  
    delay(5);  
    lcden=1;  
    delay(5);  
    lcden=0;  
}
```



```
void init()
```

```
{
```

```
    lcden=0;
```

```
    write_com(0x38);
```

```
    write_com(0x0e);
```

```
    write_com(0x06);
```

```
    write_com(0x01);
```

```
}
```

```
void main()
```

```
{
```

```
    init();
```

```
    write_com(0x80);
```

```
    write_data('O');
```

```
    write_com(0x81);
```

```
    write_data('K');
```

```
    while(1);
```

```
}
```

```
// LCD初始化子程序
```

```
// 主程序在LCD开头显示 “OK”
```

8.5 A/D、D/A转换器与单片机的接口技术

- 单片机处理的是数字量，然而在单片机的实时控制和智能仪表等应用系统中，被控制量或被测量对象的有关参量往往是一些连续变化的模拟量，如温度、压力、流量、速度等物理量，这些模拟量必须装换成数字量后才能输入到计算机进行处理。
- 计算机处理的结果，也常常需要转换为模拟信号，驱动相应的执行机构，实现对被控对象的控制。如果输入是非电的模拟信号，还需通过传感器转换为电信号并加以放大。
- 这是就需要解决单片机与A/D和D/A的接口技术问题。

8.5 A/D、D/A转换器与单片机的接口技术

◇模数（A/D）转换接口

◆A/D转换器

模/数转换：（A/D，Analog to Digit），实现模拟量转换为数字量的过程称为“量化”，也称为模/数转换。

A/D转换器：实现模/数转换的设备称为A/D转化器或ADC。

A/D转换电路分类：根据转换原理可以分为逐次逼近式、双积分式、并行式、计数式等。

8.5 A/D、D/A转换器与单片机的接口技术

◆A/D转换的主要技术指标

(1) 转换时间

A/D转换时间就是A/D转换器完成一次模拟量变换为数字量所需时间。

(2) 分辨率

A/D转换器的分辨率是指转换器对输入电压微小变化响应能力的度量，习惯上以输出的二进制位数或者BCD码位数表示。

(3) 转换精度

A/D转换器转化精度反映了一个实际A/D转换器在量化值上与一个理想A/D转换器进行模/数转换的差值。转换精度可表示成绝对误差或相对误差，与一般测试仪表的定义相似。

8.5 A/D、D/A转换器与单片机的接口技术

◆ 典型A/D转换芯片ADC0809

(1) 逐次逼近式A/D转换器转换原理

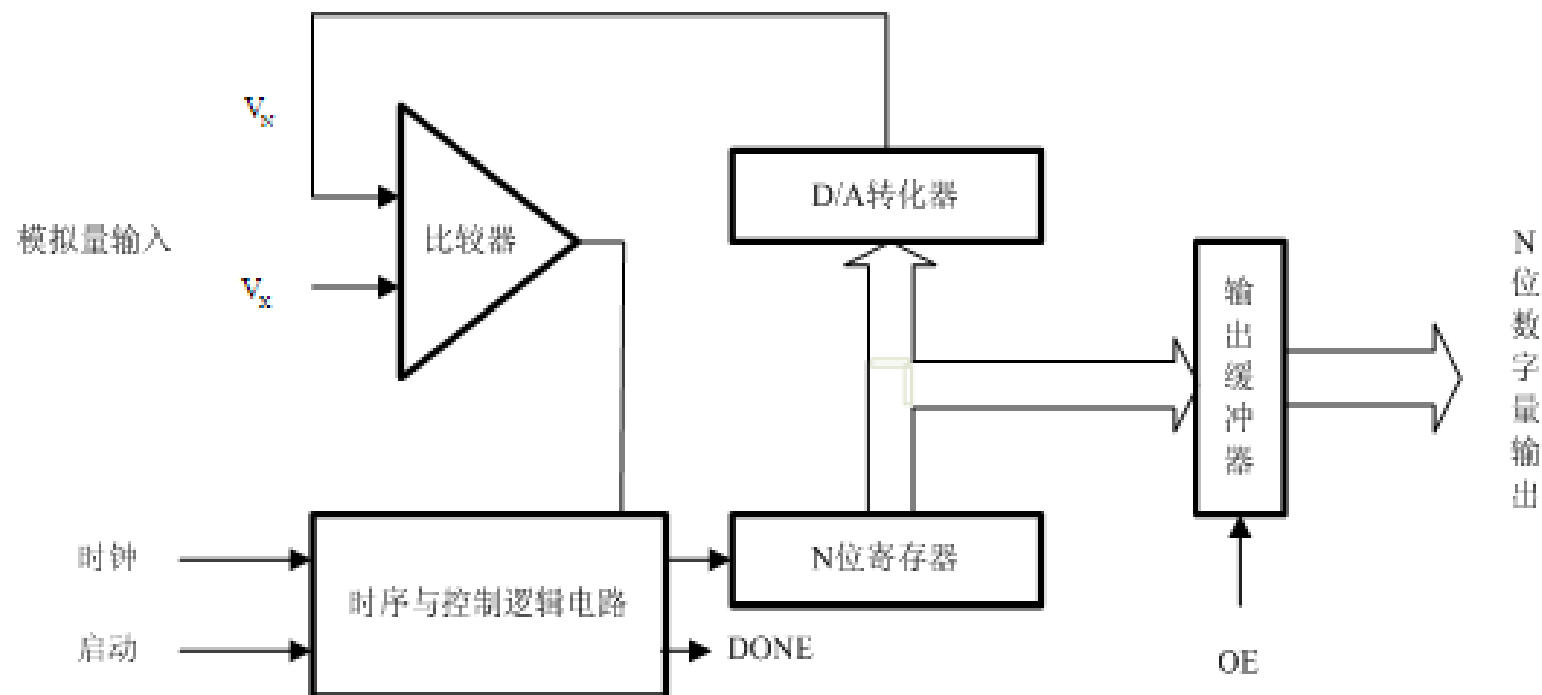


图 8-25 逐次逼近式 A/D 转换原理图

8.5 A/D、D/A转换器与单片机的接口技术

◆典型A/D转换芯片ADC0809

(2) ADC0809

ADC0809是美国国家半导体公司生产的一种8路模拟量输入8位数字量输出的逐次逼近型A/D器件。

主要技术指标和特性：

- 分辨率为8位；
- 转换时间取决于芯片时钟频率，当CLK=500KHz时，转换时间为128us；
- +5V的单一电源供电；
- 模拟输入电压范围为单极性0~+5V；
- 具有可控三态输出锁存器；
- 启动转换控制为脉冲式（正脉冲），上升沿使内部所有寄存器清“0”，下降沿使A/D转换开始。

ADC0809结构框图

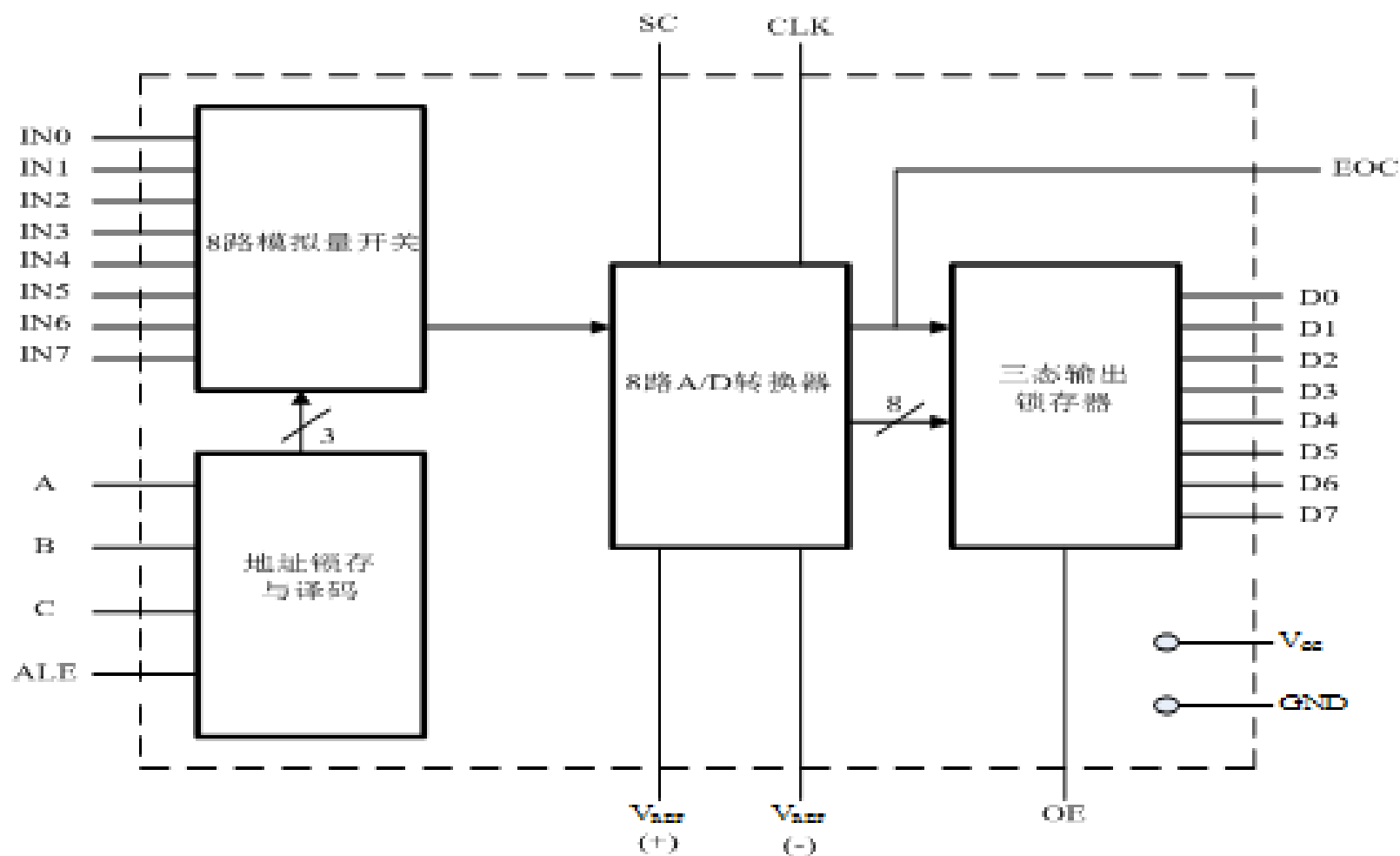
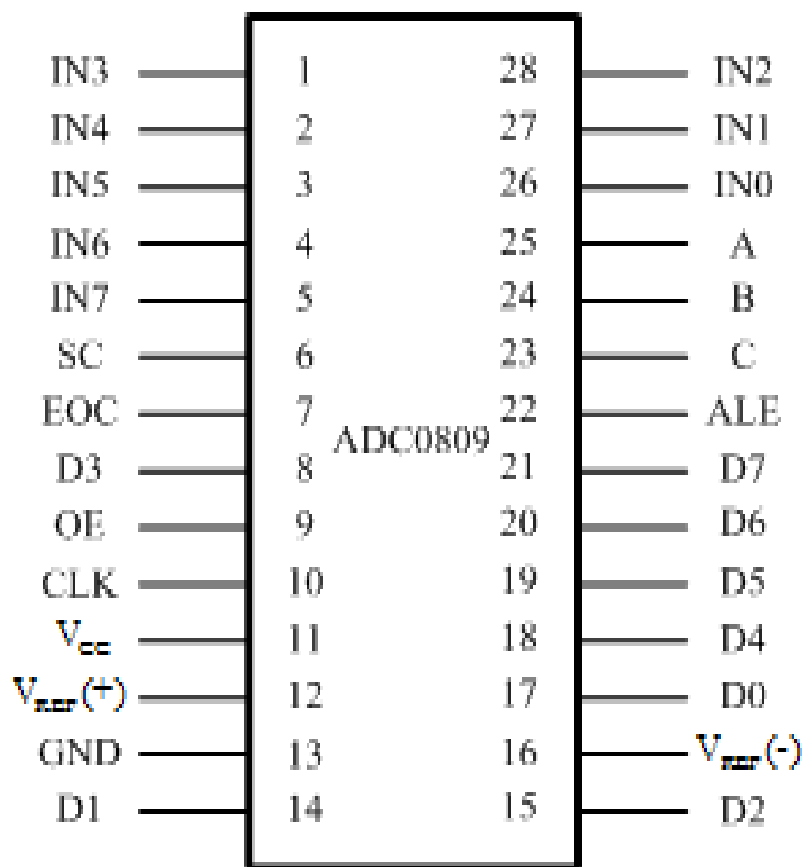


图 8-26 ADC0809 结构框图

ADC0809引脚图



IN0~IN7:8路模拟量的输入端。

D0~D7:A/D转化后的数据输出端

A、B、C: 模拟通道地址选择端

VREF(+)、VREF(-): 基准参考电压

CLK: 时钟信号输入端。

ALE: 地址锁存允许信号。

SC: 启动转换信号。

EOC: 转换结束信号。

OE: 输出允许信号。

图 8-27 ADC0809 引脚图

通道选择的地址编码如表8-8所示

表 8-8 通道选择的地址编码

地址编码			被选中的通道
C	B	A	
0	0	0	IN0
0	0	1	IN1
0	1	0	IN2
0	1	1	IN3
1	0	0	IN4
1	0	1	IN5
1	1	0	IN6
1	1	1	IN7

◆单片机与ADC0809的接口电路

图8-28是ADC0809与AT89S52的连接示意图。由于ADC0809片内无时钟，可利用AT89S52提供的地址锁存允许信号ALE经D触发器二分频得到。因为0809内部具有地址锁存器,也可以将A、B、C直接和P0.0、P0.1、P0.2相连。

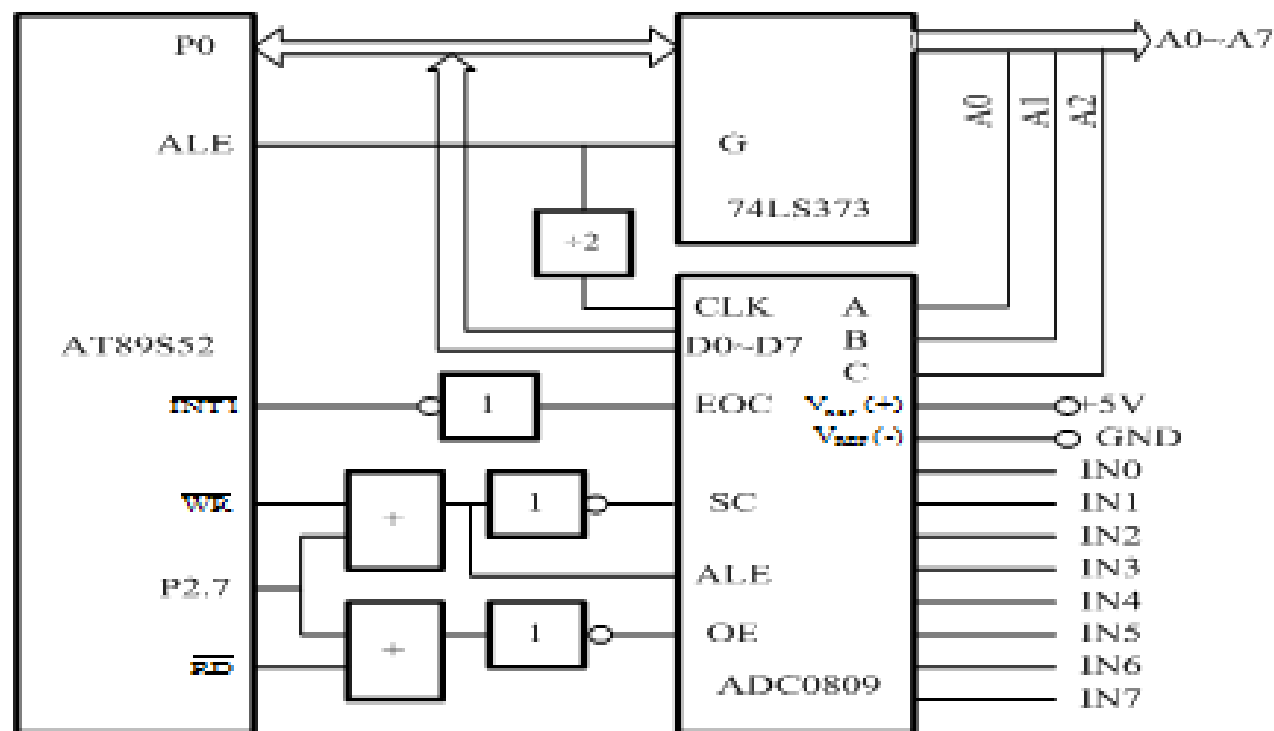


图 8-28 ADC0809 与 AT89S52 的连接

假设在以图8-28所示电路为基础的测控系统中，巡回检测一遍8路模拟量输入，采用中断方式将读数依次存放在片外数据存储器A0H~A7H单元, 其程序如下：汇编语言程序：

```
ORG 000H
LJMP MAIN
ORG 0013H
LJMP INT11
ORG 0100H
MAIN:MOV  R0, #0A0H      ;数据存储区首址
      MOV  R2, #08H      ;8路计数值
      SETB IT1           ;边沿触发方式
      SETB EA            ;中断允许
      SETB EX1           ;允许外部中断1中断
      MOV  DPTR, #7FF8H  ;A/D转换器地址
      MOV  A, #00H       ;指向0通道
LOOP: MOVX  @DPTR, A      ;启动A/D转换
      HERE: SJMP  HERE   ;等待中断
      ORG 0200H         ;中断服务程序
INT11:MOVX  A, @DPTR     ; 读取转换结果
      MOVX  @R0, A       ;存数
      INC  DPTR          ;指向下一个模拟通道
      INC  R0            ;指向数据存储区下一个单元
      DJNE R2, NEXT      ; 8路没采集完转
      CLR  EA
      CLR  EX1
      RETI
NEXT: MOV  @DPTR, A      ; 启动下一路转换
      RETI
```

C语言程序:

```
#include <reg52.h>
#include <absacc.h>
#define IN0=XBYTE[0x7ff8]
#define AD0=PBYTE[0xa0]
#define uchar unsigned char
uchar xdata *addz;      //定义指向通道的指针变量
uchar pata adccdata[8]; //定义存放转换结果的数组
uchar i;
void main(void)
{
    IT1=1;
    EA=1;
    EX1=1;
    i=0;
    addz=&IN0;  //通道指针初始化
    *addz=i;    //启动0通道A/D转换
    while(1);  //等待中断
}
```




```
void init1 ( void) interrupt 2  //中断函数
```

```
{
```

```
    adcddata[i]=*addz;  // 读取转换结果，并完成存储
```

```
    i++;
```

```
    addz++;
```

```
    if(i<8)                //8路未转换完
```

```
        { *addz=i;}        //启动下一路转换
```

```
        else
```

```
        {
```

```
            EA=0;EX1=0;
```

```
        }
```

```
            //8路转换完，关中断
```

```
}
```

8.5 A/D、D/A转换器与单片机的接口技术

◇数模（D/A）转换接口

将数字量转换为模拟量的过程称为数/模转换（D/A，Digit to Analog），实现D/A转换的设备称为D/A转换器或DAC。

◆D/A转换器的主要技术指标

1) 分辨率

分辨率表示对输入的最小数字量信号的分辨能力

2) 建立时间

也可称之为D/A转换速度。

3) 转换精度

精度参数用于表明D/A转换的精确程度，一般用误差大小表示。

8.5 A/D、D/A转换器与单片机的接口技术

◆ 典型D/A转换芯片DAC0832

美国国家半导体公司**DAC0832**芯片是具有**2**个输入数据寄存器的**8**位**DAC**，芯片为**20**引脚，双列直插式封装，能直接与**AT89S52**单片机直接相连接，其主要特性如下：

- 1) 分辨率为**8**位；
- 2) 电流输出，稳定时间为**1 μ s**；
- 3) 可双缓冲输入、单缓冲输入或直接数字输入；
- 4) 单一电源供电（**+5V~+15V**）；
- 5) 低功耗，**20mW**。

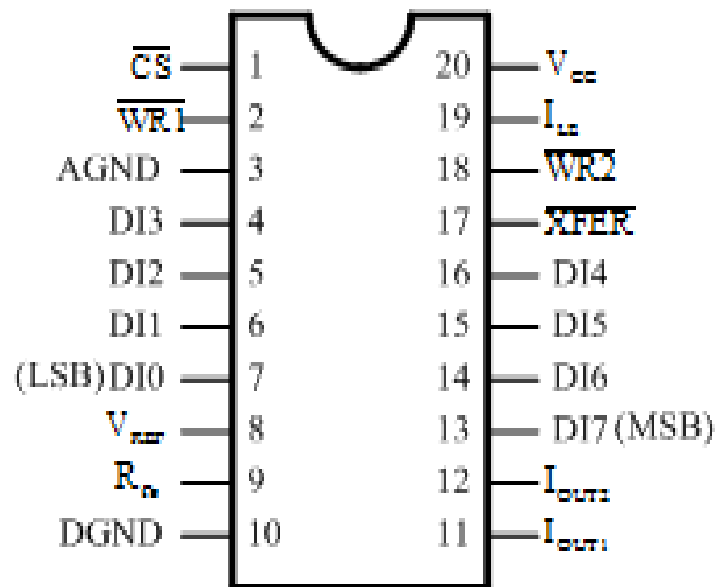


图 8-30 DAC0832 的引脚

DAC 0832逻辑结构如图8-29所示。

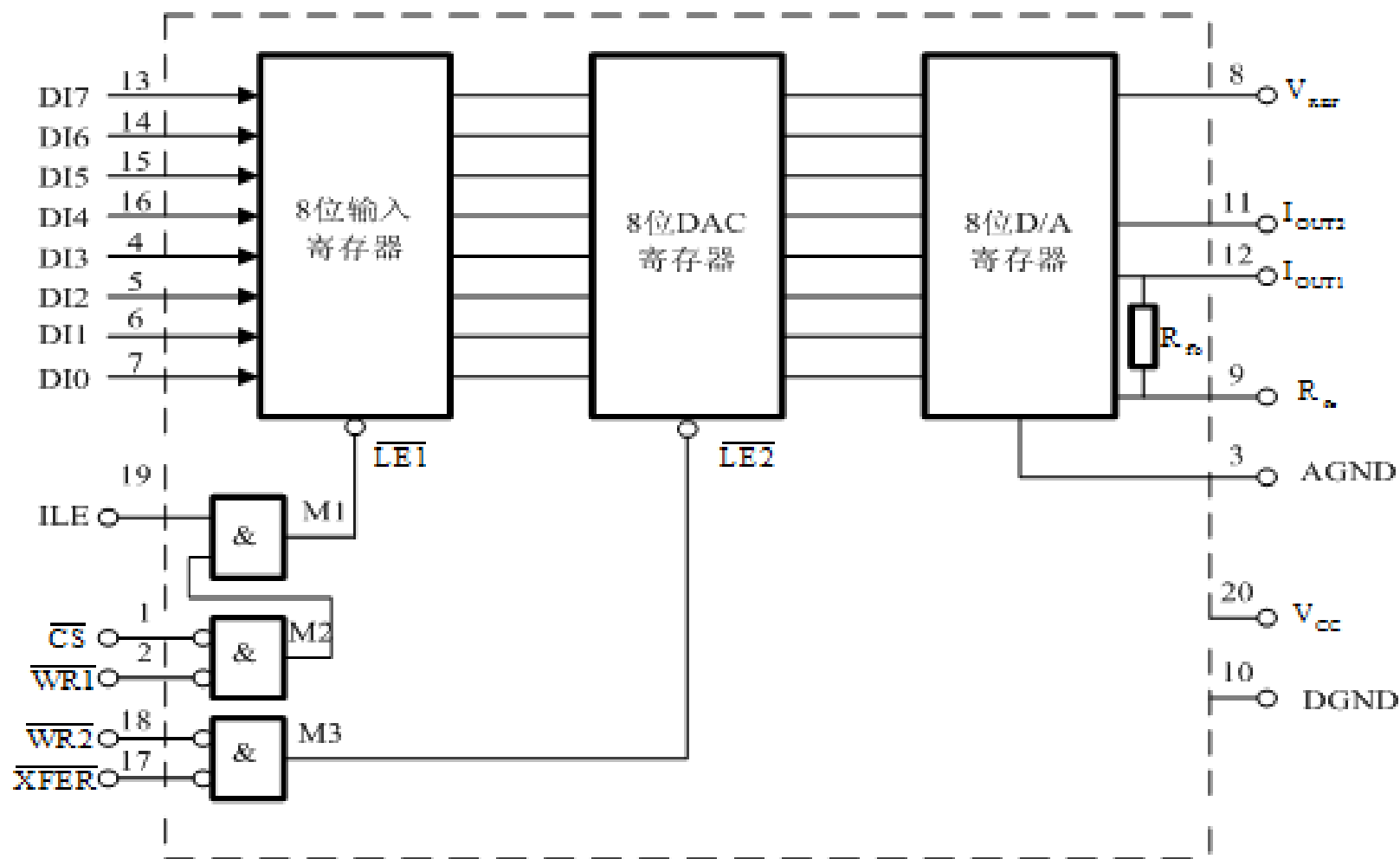


图 8-29 DAC0832 的逻辑结构

8.5 A/D、D/A转换器与单片机的接口技术

◆ AT89单片机与DAC0832的接口电路

根据对**DAC0832**的输入寄存器和**DAC**寄存器的不同控制方法，可有**3种**工作方式：

- ① 单缓冲方式
- ② 双缓冲方式
- ③ 直通方式

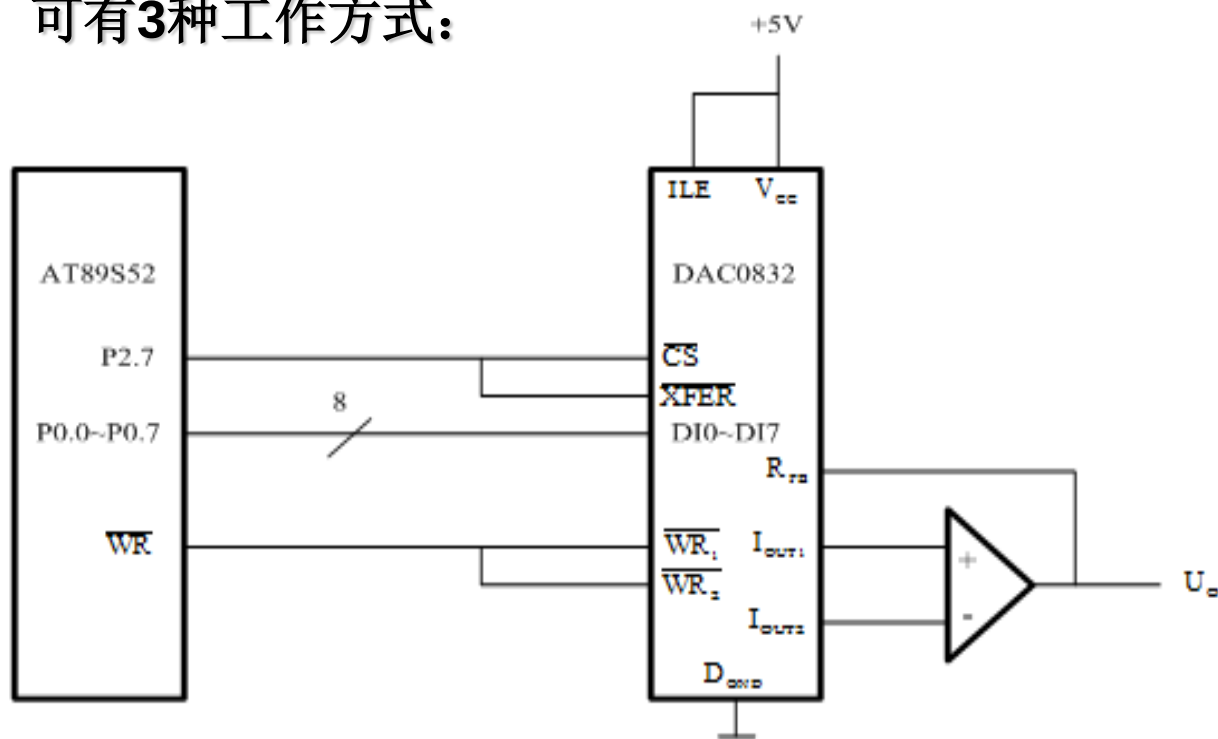


图 8-31 DAC0832 单缓冲器方式应用

(1) 单缓冲方式接口电路及应用

产生锯齿波程序清单：

```
START: MOV  DPTR, #7FFFH      ; 设置D/A口地址
        MOV  A, #00H          ; 输入数字量初始值00H
到A
LOOP:  MOVX  @DPTR, A          ; 输出对应于A内容的模拟量
        INC  A                 ; 修改A
的内容
        AJMP LOOP             ; 循环
```

C语言程序：

```
#include<reg52.h>
#include<absacc.h>
#define DAC0832 XBYTE[0x7fff]
void main()                //主函数
{
    unsigned char i;
    while(1)
    {
        for(i=0;i<255;i++)
        {
            DAC0832=i;
        }
    }
}
```

产生方波程序清单：

START:	MOV DPTR, #7FFFH	； 设置D/A口地址
LOOP:	MOV A, #0FFH	； 给A送最大值
	MOVX @DPTR, A	； D/A输出相应模拟量
	ACALL 延时子程序	； 延时
	MOV A, #00H	； 给A送最小值
	MOVX @DPTR, A	； D/A输出相应模拟量
	ACALL 延时子程序	； 延时
	AJMP LOOP	； 循环

C语言程序:

```
#include<reg52.h>
#include<absacc.h>
#define DAC0832 XBYTE[0x7fff]
void delay(unsigned int i)    // 延时函数
{
    while(i--);
}

void main()                  // 主函数
{
    unsigned char i;
    while(1)
    {
        for(i=0;i<255;i++)
        {
            DAC0832=0xff;
            delay(15);
            DAC0832=0;
            delay(15);
        }
    }
}
```


(2) 双缓冲方式接口电路及应用

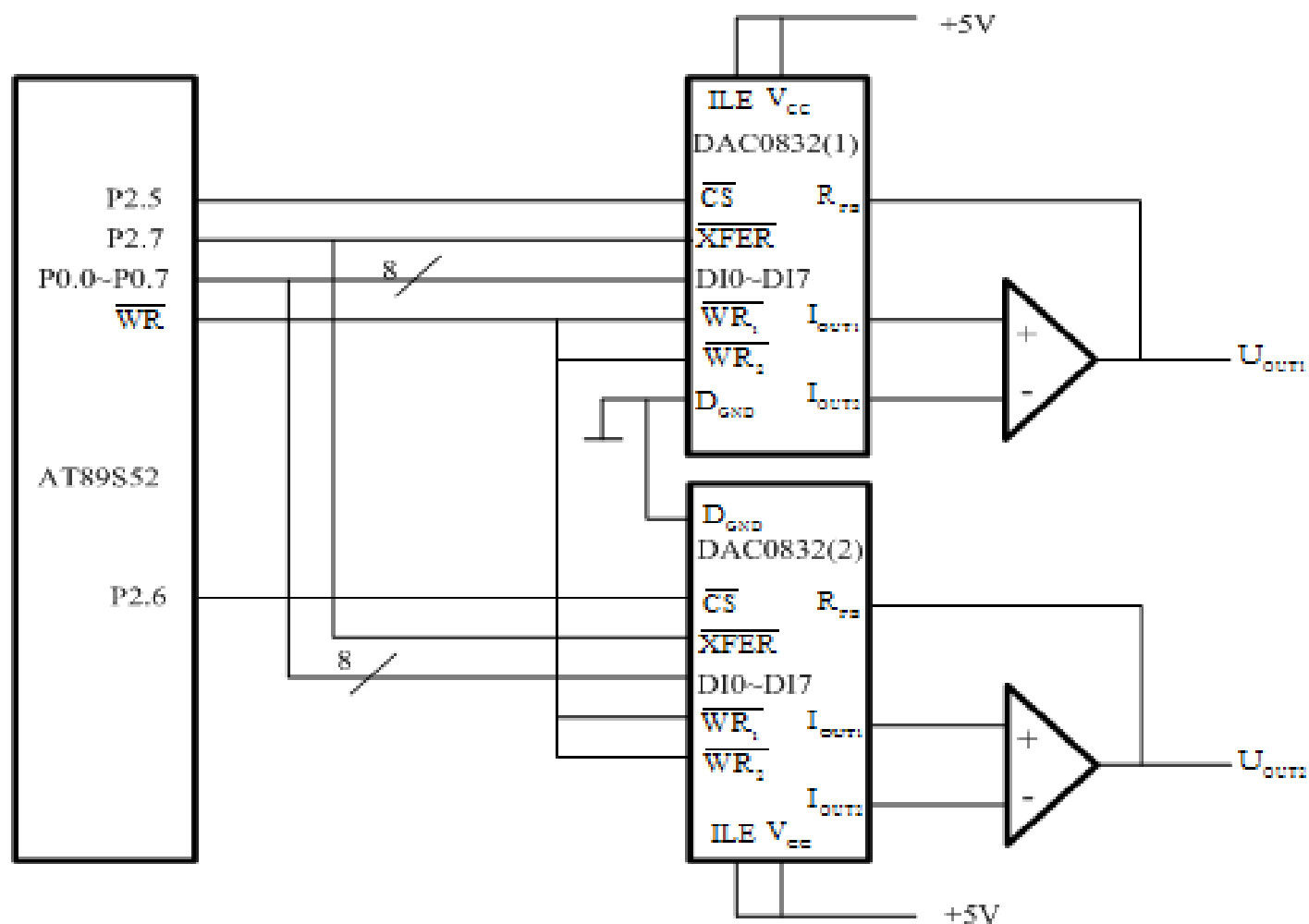


图 8-32 DAC0832 双缓冲器方式应用

在图8-33中每一路模拟量输出需一片DAC0832。DAC0832(1)输入锁存器地址为0DFFFH，DAC0832(2)输入锁存器的地址为0BFFFH。DAC0832(1)和DAC0832(2)的第二级寄存器地址同为7FFFH。根据图8-32，编写程序实现两路D/A同步转换输出。

程序清单如下：

```
ORG 0100H

MOV DPTR, #0DFFFH    ; 指向DAC0832(1)

MOV A, #DATA1        ; DATA1送入DAC0832(1)中锁存

MOVX @DPTR, A

MOV DPTR, #0BFFFH    ; 指向DAC0832(2)

MOV A, #DATA2        ; DATA2送入DAC0832(2)中锁存

MOVX @DPTR, A

MOV DPTR, #7FFFH     ; 给0832(1)和(2)提供信号

MOVX @DPTR, A        ; 同时完成D/A转换输出

END
```



思考：怎样产生三角波和正弦波？

C语言程序:

```
#include<reg52.h>
#include<absacc.h>
#define DAC0832 (1)  XBYTE[0xdfff]
#define DAC0832 (2)  XBYTE[0xbfff]
#define DAC0832 (3)  XBYTE[0x7fff]
void main()                //主函数
{
    DAC0832(1)=data1;
    DAC0832(2)=data2;
    DAC0832(3)= data; //随便写入一个数，打开第二级寄存器，
    同时启动D/A转换
}
```



本章小结